DATA 260 Homework 6

MongoDB Documents and Memory Systems

Student Name: Priyank Mehta

SJSU ID: 019152562

GitHub Repository: https://github.com/priyankmehta5/data260-2562

1. Personal Configuration

This report documents the MongoDB document API, conversational memory system, memory-cost experiments, and file-based memory system added to the municipal transit incident application.

Value	Configuration
SID4	2562
PORT_BASE	8762
PREFIX	s2562
SEED	2562
VERIFY_SEED	262562
DOMAIN_ID	2 - Municipal transit incidents
Local model	qwen3:1.7b
Embedding model	sentence-transformers/all-MiniLM-L6-v2
Database	MongoDB s2562_doc
Hardware	Windows development computer, Python 3.12
Verified commit	d923625e48af8558bc75cfbe410f5550f39bdaef

2. Results Summary

The document API supports validated create, retrieve, update, and delete operations in MongoDB. The memory API stores messages, summaries, and episodic facts in the same database. The final smoke test confirmed MongoDB availability and a successful POST request to /api/chat.

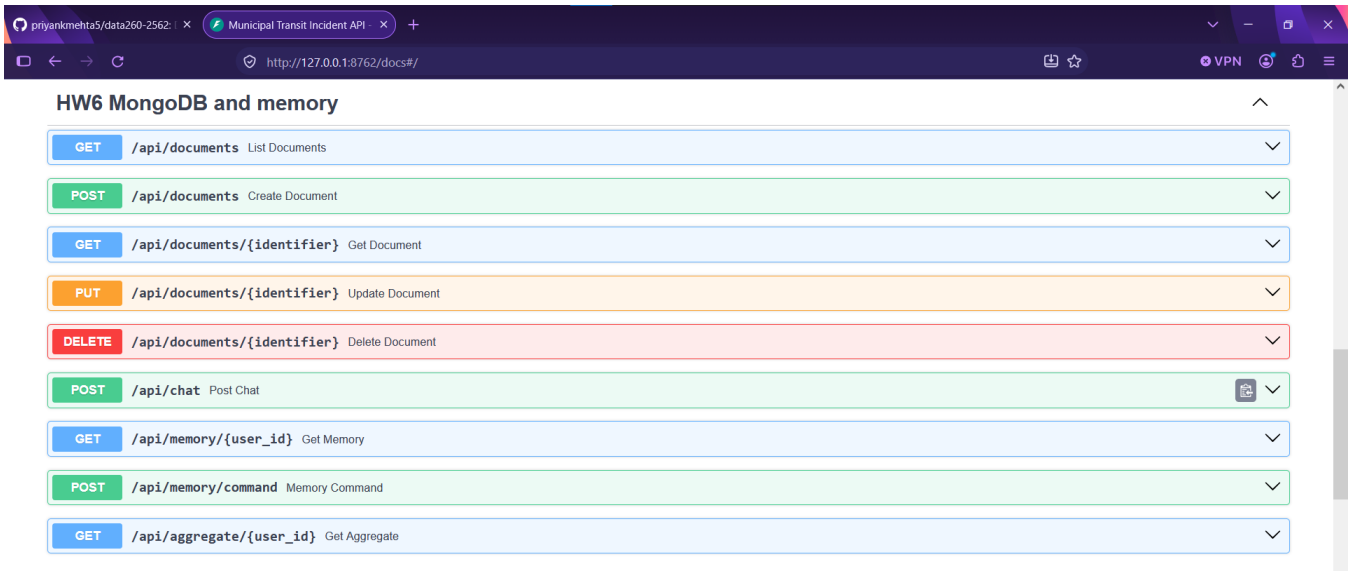
Setting	Prompt tokens/turn	Latency (ms)	Probe quality	10,000 conversations
SHORT_TERM_N=4	26.8	1.001	1.0	$11.04
SHORT_TERM_N=12	62.8	1.000	1.0	$21.84
SHORT_TERM_N=30	97.0	1.001	1.0	$32.10
eviction_last_6	37.0	1.000	3.0	$14.10

The eviction policy was the best measured setting because it reduced prompt tokens and improved the deterministic probe score. SHORT_TERM_N=30 was the highest-cost setting because it retained the largest context window. The cost estimates use $1 per million input tokens and $5 per million output tokens.

3. Evidence

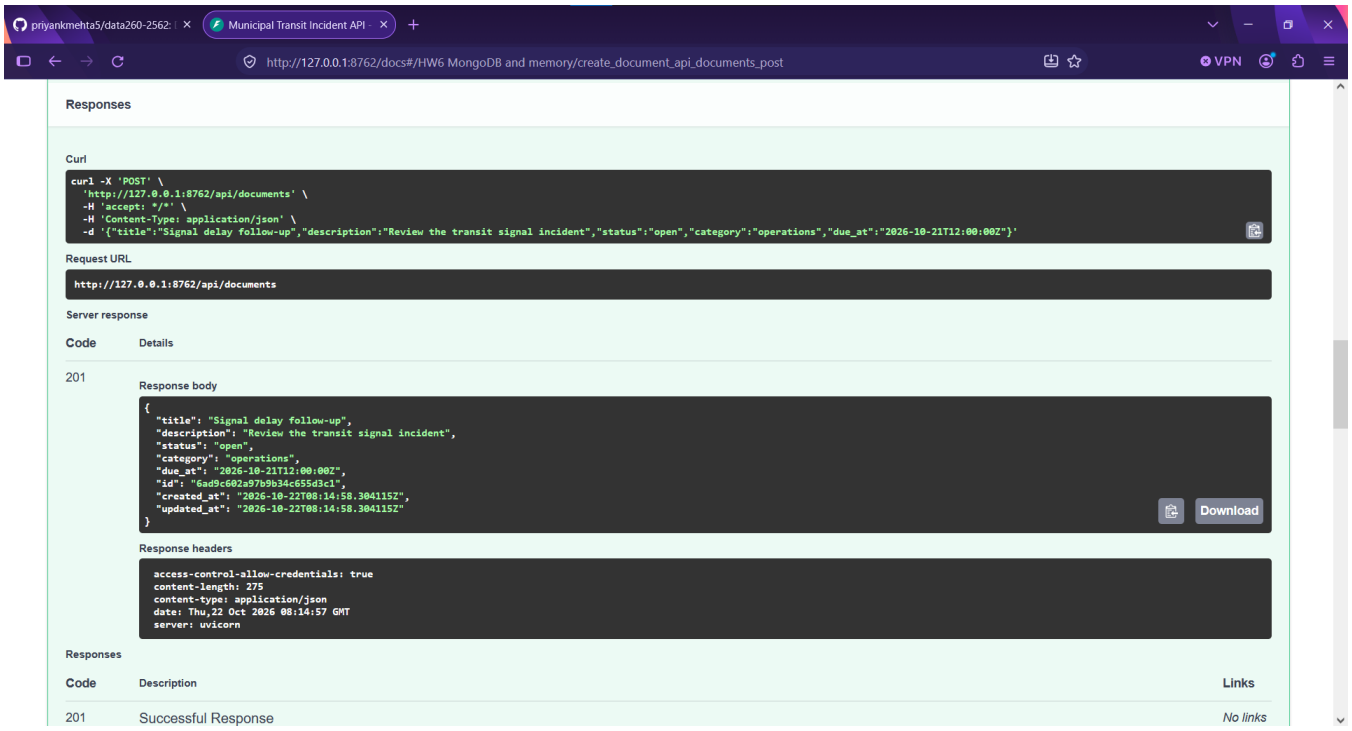
Part 1 - MongoDB Document API

Part 1 - HW6 document API endpoints



Part 1 - HW6 document API endpoints

Part 1-II - Creating a MongoDB document with FastAPI



Part 1-II - Creating a MongoDB document with FastAPI

Part 1-I - MongoDB document schema and validation rules

hw06_memory.py U

data260-2562 > backend > hw06_memory.py > MongoStore > online

```python
class MongoStore:
    def __init__(self, uri: str | None = None, fallback: MemoryStore | None = None):
        self.fallback = fallback or MemoryStore()
        self.client = None
        self.db = None
        if MongoClient is not None:
            try:
                self.client = MongoClient(uri or os.getenv("MONGODB_URI", "mongodb://127.0.0.1:27017"), serverSelectionTimeoutMS=700)
                self.client.admin.command("ping")
                self.db = self.client[DB_NAME]
                self._ensure_indexes()
            except Exception:
                self.client = None
                self.db = None

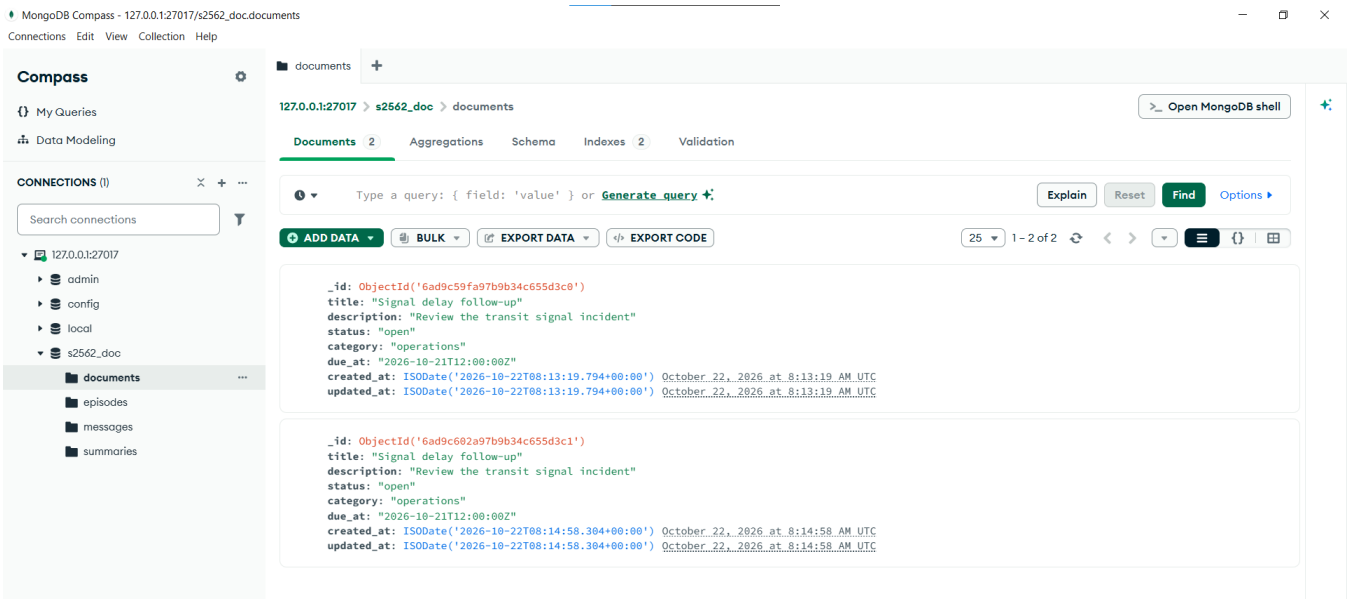
    @property
    def online(self) -> bool:
        return self.db is not None

    def _ensure_indexes(self):
        validators = {
            "documents": {"$jsonSchema": {"bsonType": "object", "required": ["title", "status", "category", "due_at", "created_at", "upda
            "messages": {"$jsonSchema": {"bsonType": "object", "required": ["user_id", "role", "content", "created_at"], "properties": {"
            "summaries": {"$jsonSchema": {"bsonType": "object", "required": ["user_id", "scope", "text", "created_at"], "properties": {"s
            "episodes": {"$jsonSchema": {"bsonType": "object", "required": ["user_id", "fact", "importance", "embedding", "created_at"],
        }
        existing = set(self.db.list_collection_names())
        for name, validator in validators.items():
            if name not in existing:
                self.db.create_collection(name, validator=validator, validationLevel="strict")
        self.db.documents.create_index("created_at")
        self.db.messages.create_index([("user_id", 1), ("session_id", 1), ("created_at", -1)])
        self.db.summaries.create_index([("user_id", 1), ("scope", 1), ("created_at", -1)])
        self.db.episodes.create_index([("user_id", 1), ("created_at", -1)])

    @staticmethod
    def _mongo_id(identifier: str):
```

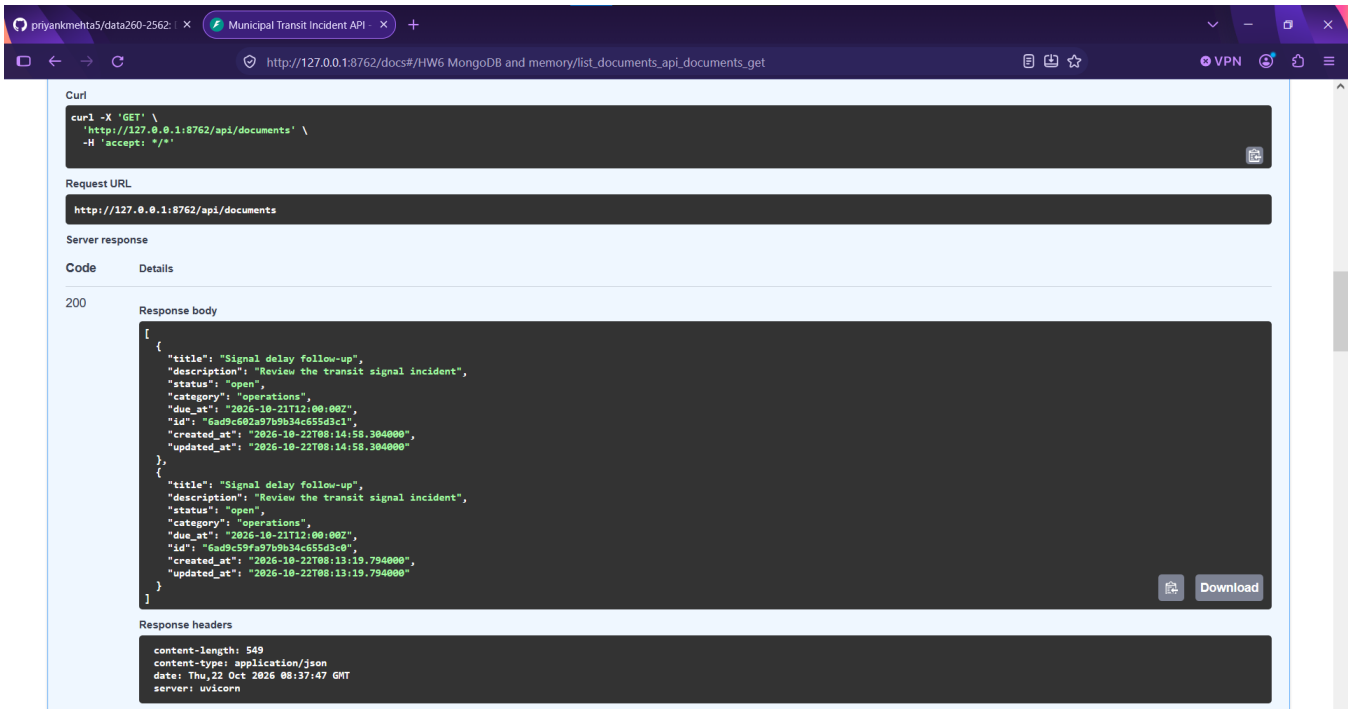
Part 1-I - MongoDB document schema and validation rules

Part 1-I - MongoDB Compass documents collection



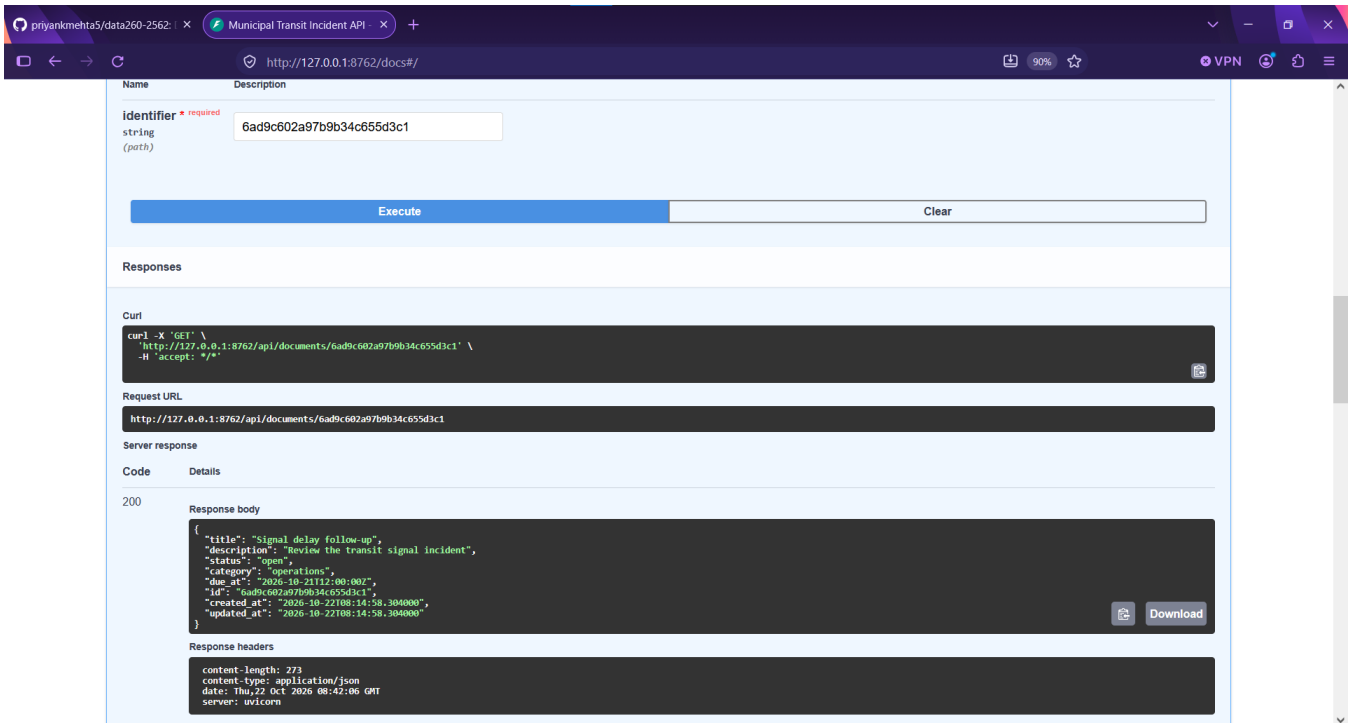
Part 1-I - MongoDB Compass documents collection

Part 1-II - Retrieving all MongoDB documents



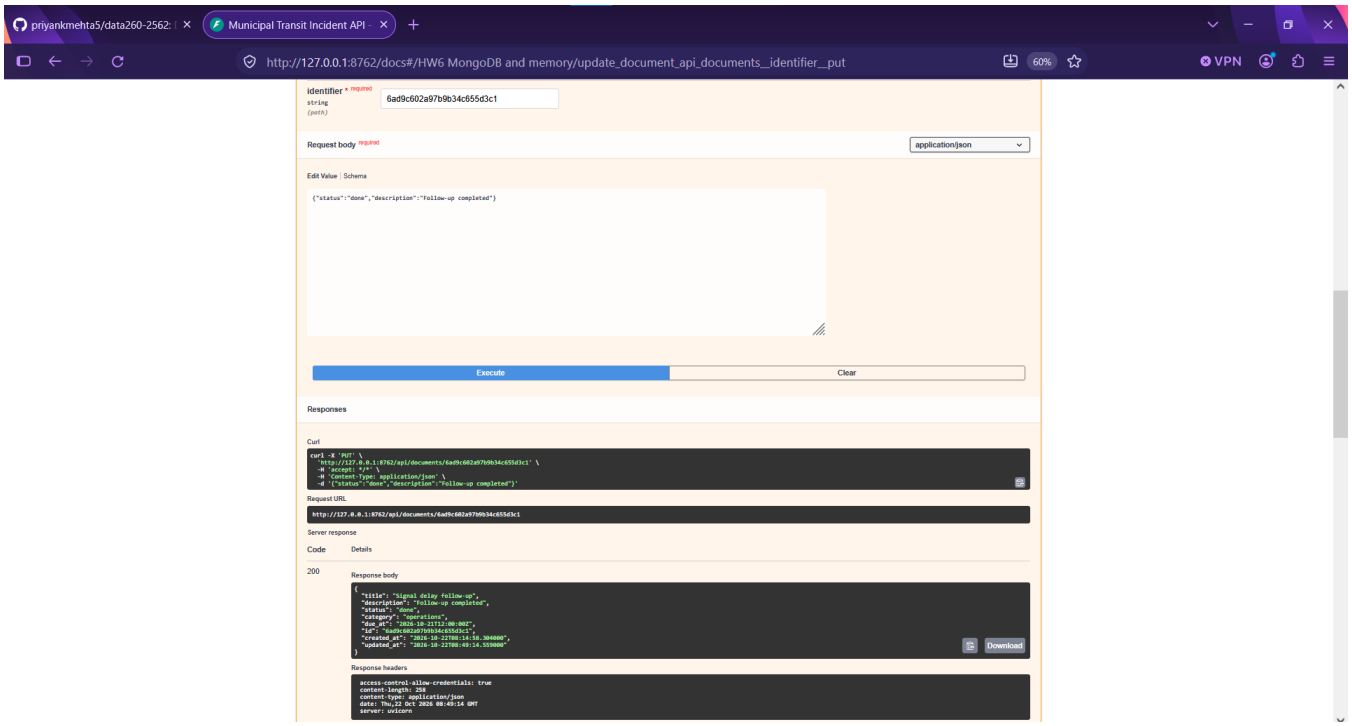
Part 1-II - Retrieving all MongoDB documents

Part 1-II - Retrieving one MongoDB document



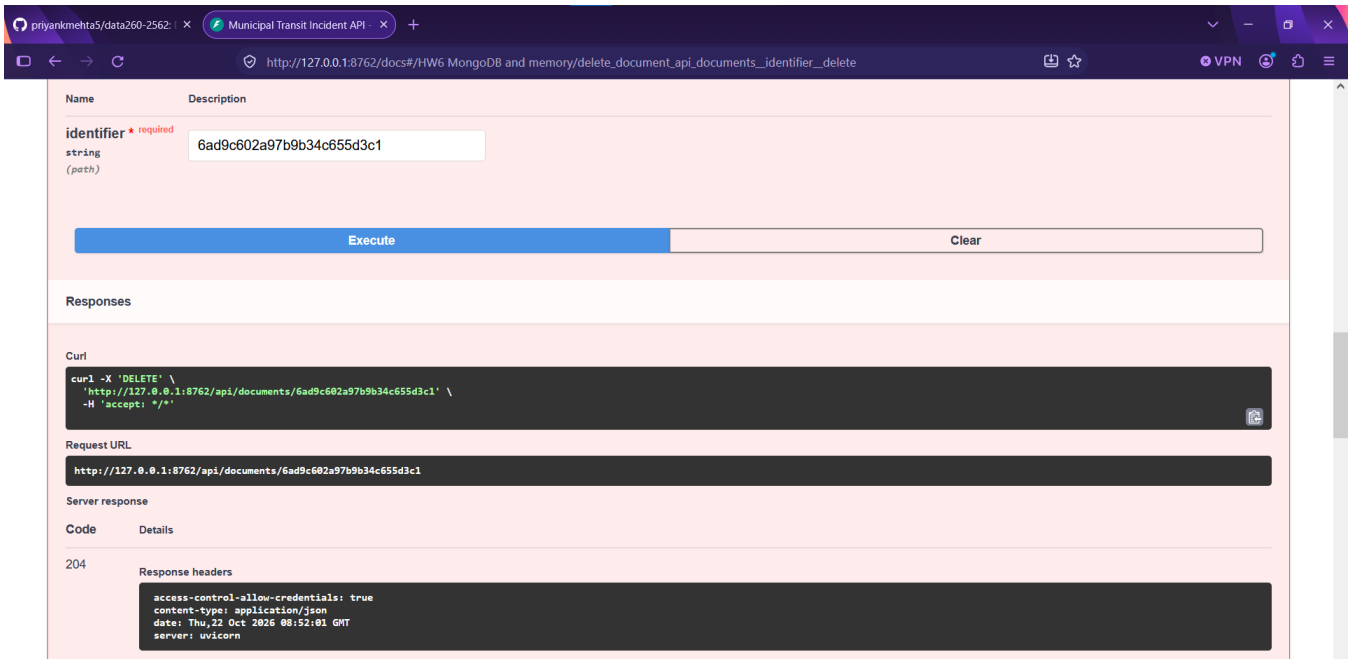
Part 1-II - Retrieving one MongoDB document

Part 1-II - Updating a MongoDB document



Part 1-II - Updating a MongoDB document

Part 1-II - Deleting a MongoDB document



Part 1-II - Deleting a MongoDB document

Part 1-III - Pydantic validation schemas

hw06_memory.py U

data260-2562 > backend > hw06_memory.py > DocumentCreate

```python
class DocumentStatus(str, Enum):
    open = "open"
    in_progress = "in_progress"
    done = "done"


class DocumentCategory(str, Enum):
    service = "service"
    safety = "safety"
    operations = "operations"
    other = "other"


class DocumentCreate(BaseModel):
    title: str = Field(min_length=1, max_length=100)
    description: str | None = None
    status: DocumentStatus = DocumentStatus.open
    category: DocumentCategory = DocumentCategory.other
    due_at: datetime

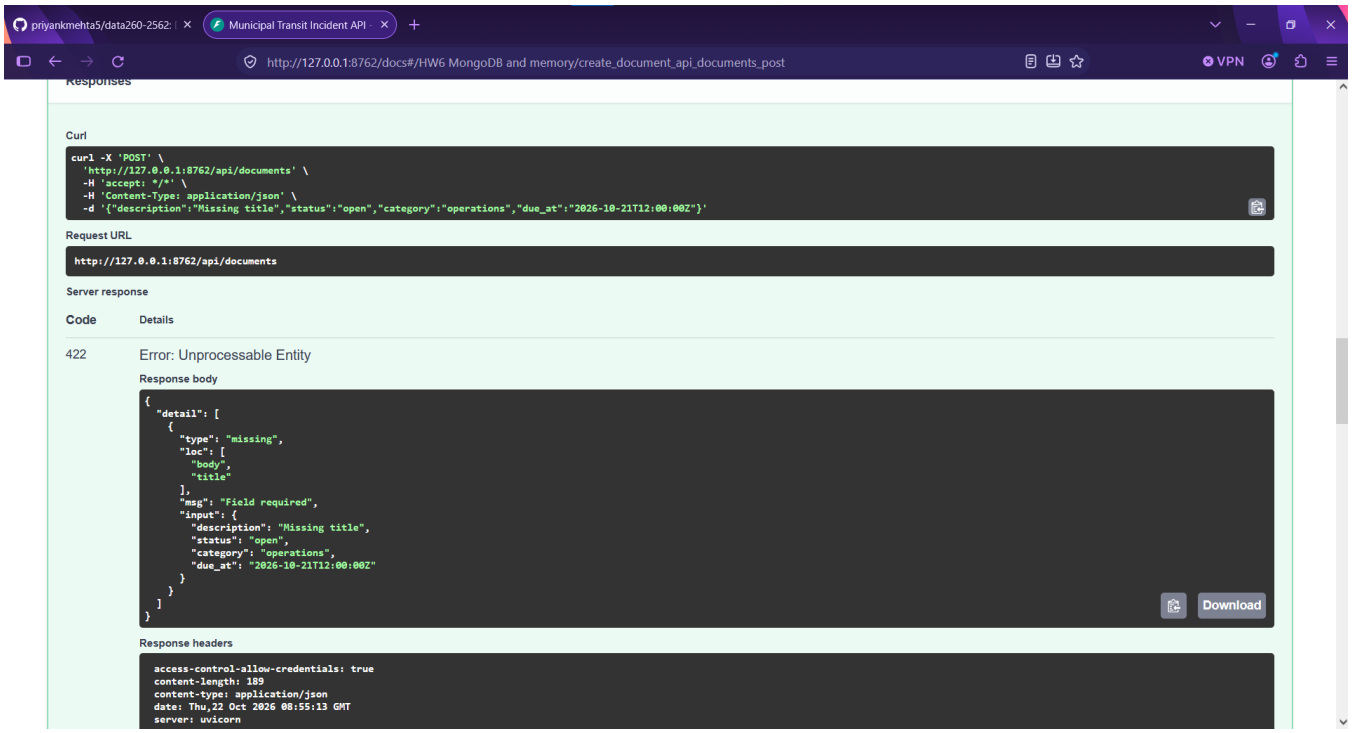
    @field_validator("title")
    @classmethod
    def clean_title(cls, value: str) -> str:
        value = value.strip()
        if not value:
            raise ValueError("title must not be blank")
        return value


class DocumentUpdate(BaseModel):
    title: str | None = Field(default=None, min_length=1, max_length=100)
    description: str | None = None
    status: DocumentStatus | None = None
    category: DocumentCategory | None = None
    due_at: datetime | None = None

```

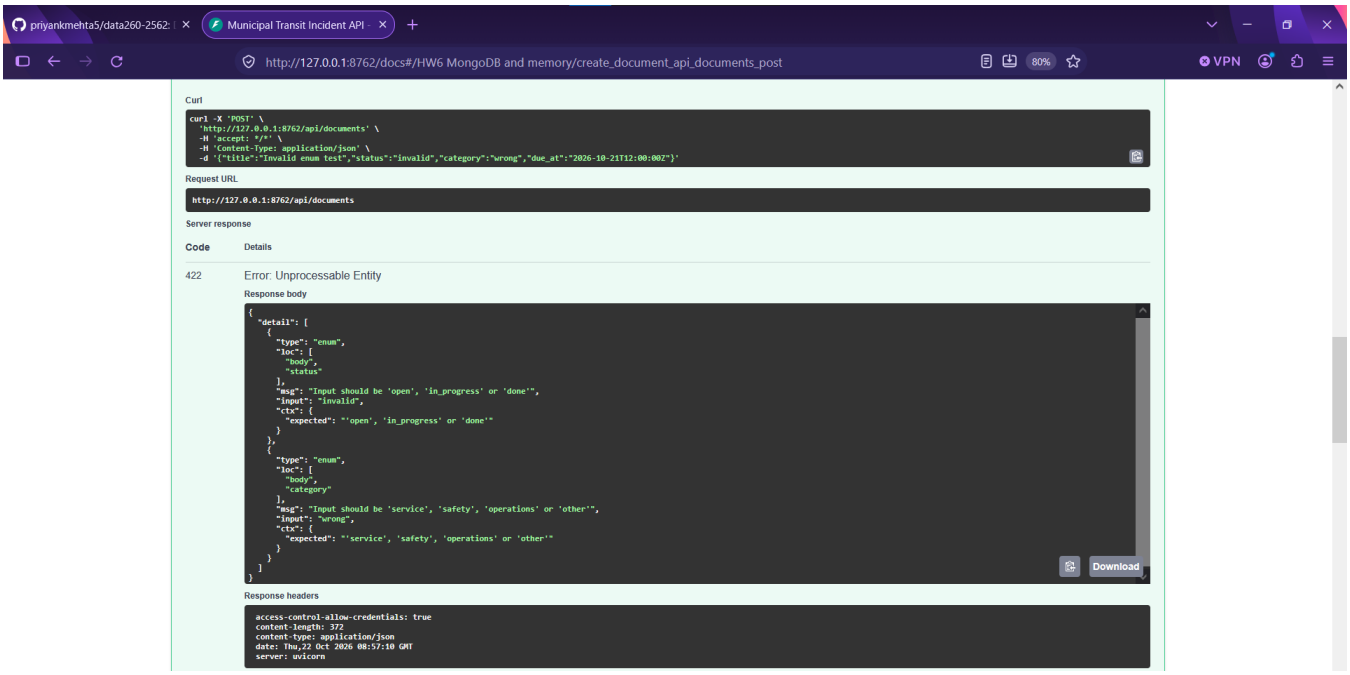
Part 1-III - Pydantic validation schemas

Part 1-III - Validation error for missing required field



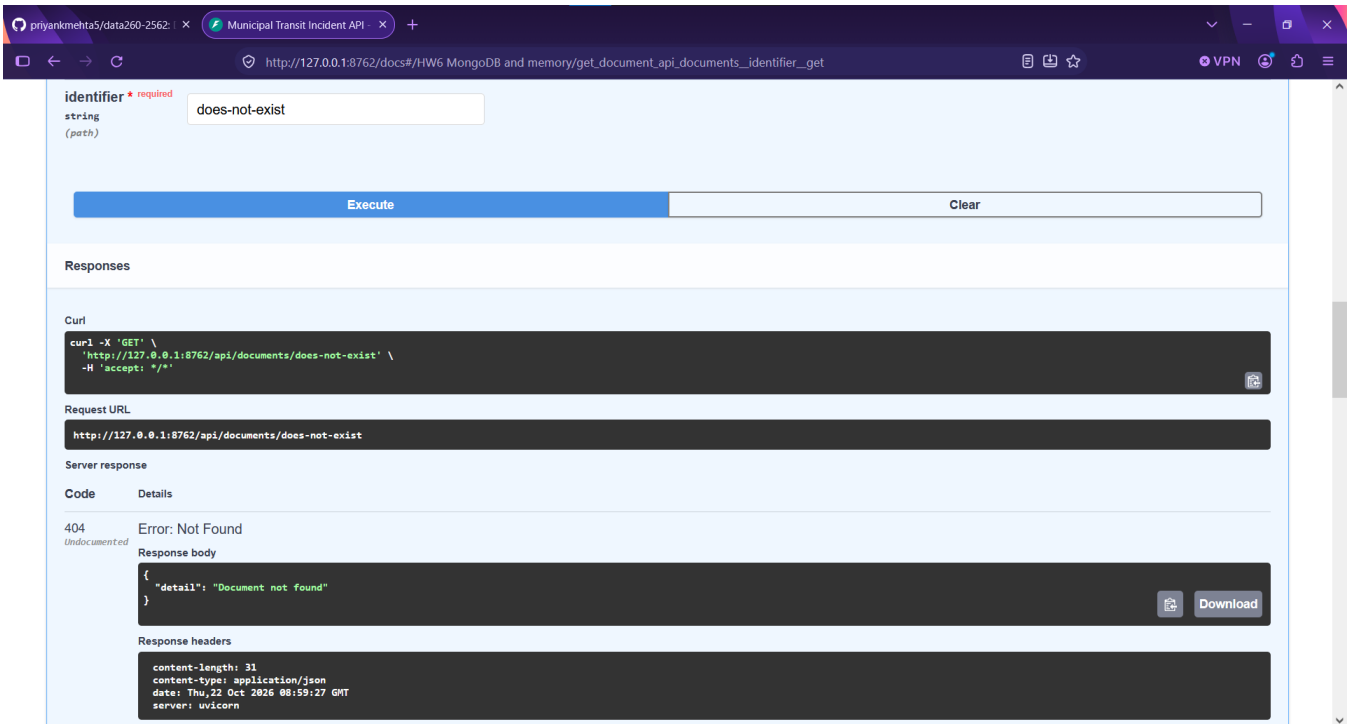
Part 1-III - Validation error for missing required field

Part 1-III - Validation error for invalid enum values



Part 1-III - Validation error for invalid enum values

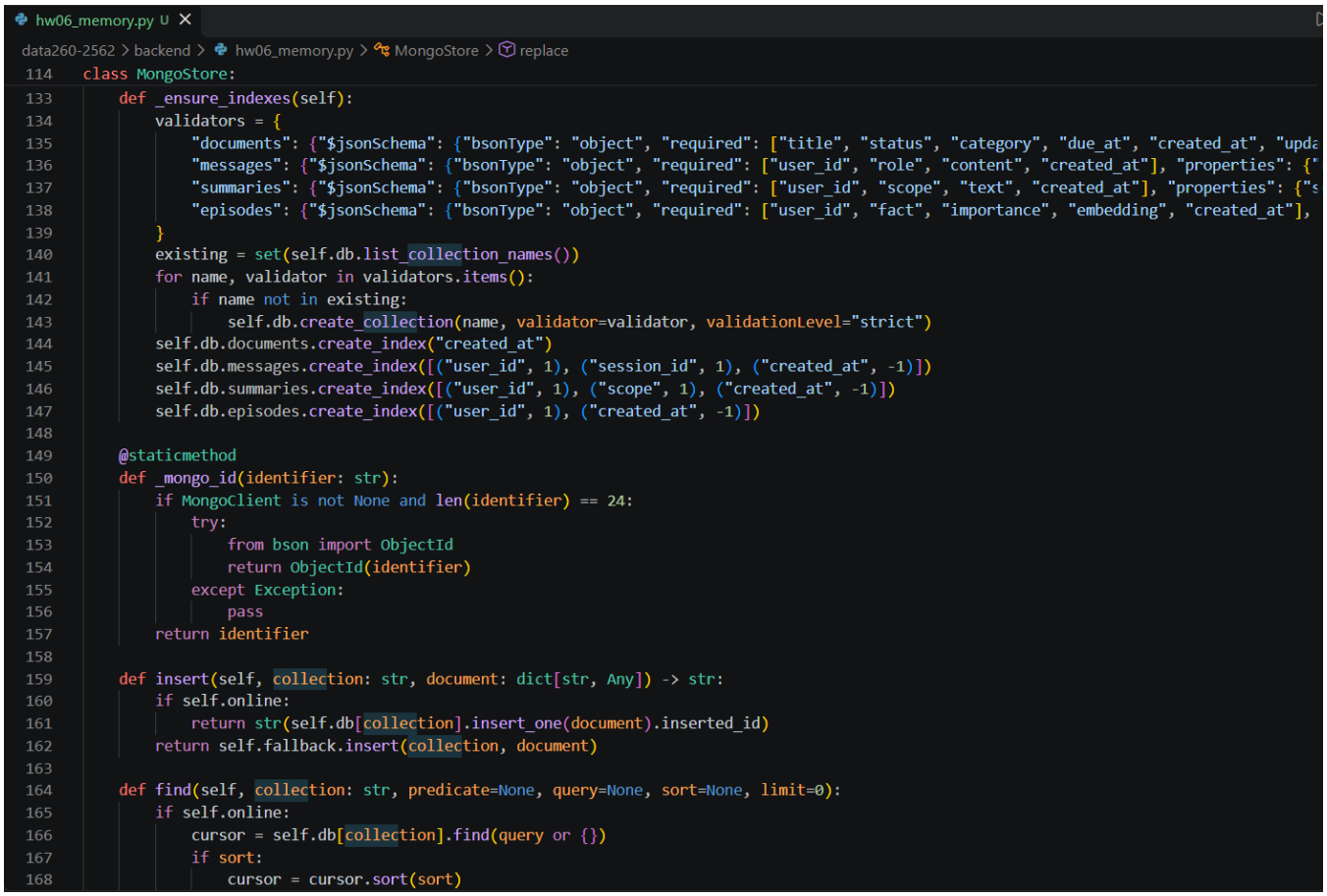
Part 1-III - Not-found error handling



Part 1-III - Not-found error handling

Part 2 - Short-, Long-, and Episodic Memory

Part 2 - MongoDB messages, summaries, and episodes collection validation



Part 2 - MongoDB messages, summaries, and episodes collection validation

Part 2 - Memory collection declarations

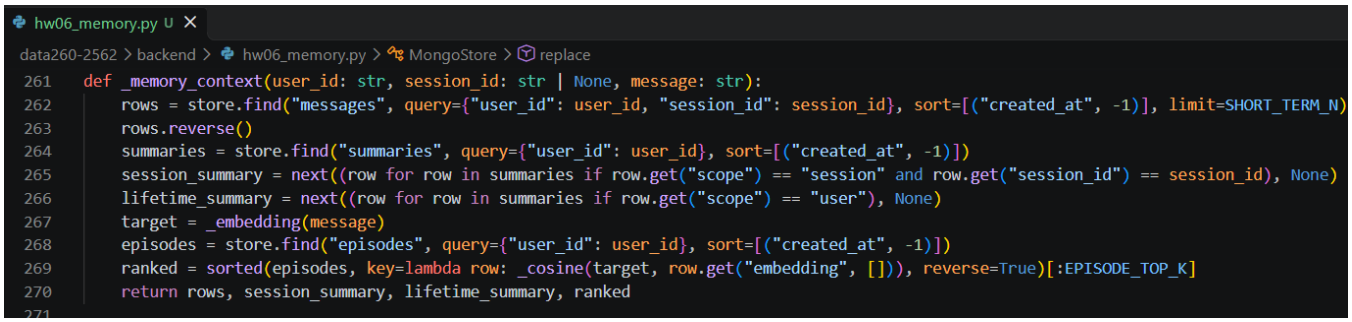
hw06_memory.py U

data260-2562 > backend > hw06_memory.py > MongoStore > replace

```
81  class MemoryStore:
82      def __init__(self):
83          self.documents: list[dict[str, Any]] = []
84          self.messages: list[dict[str, Any]] = []
85          self.summaries: list[dict[str, Any]] = []
86          self.episodes: list[dict[str, Any]] = []
```

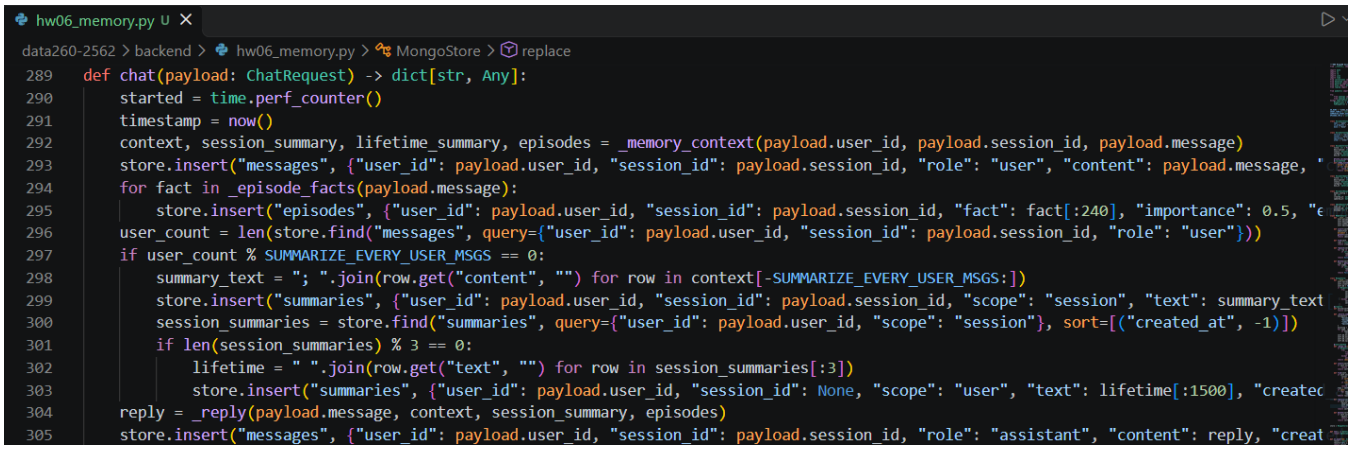
Part 2 - Memory collection declarations

Part 2 - Short-term, summary, and episodic memory retrieval



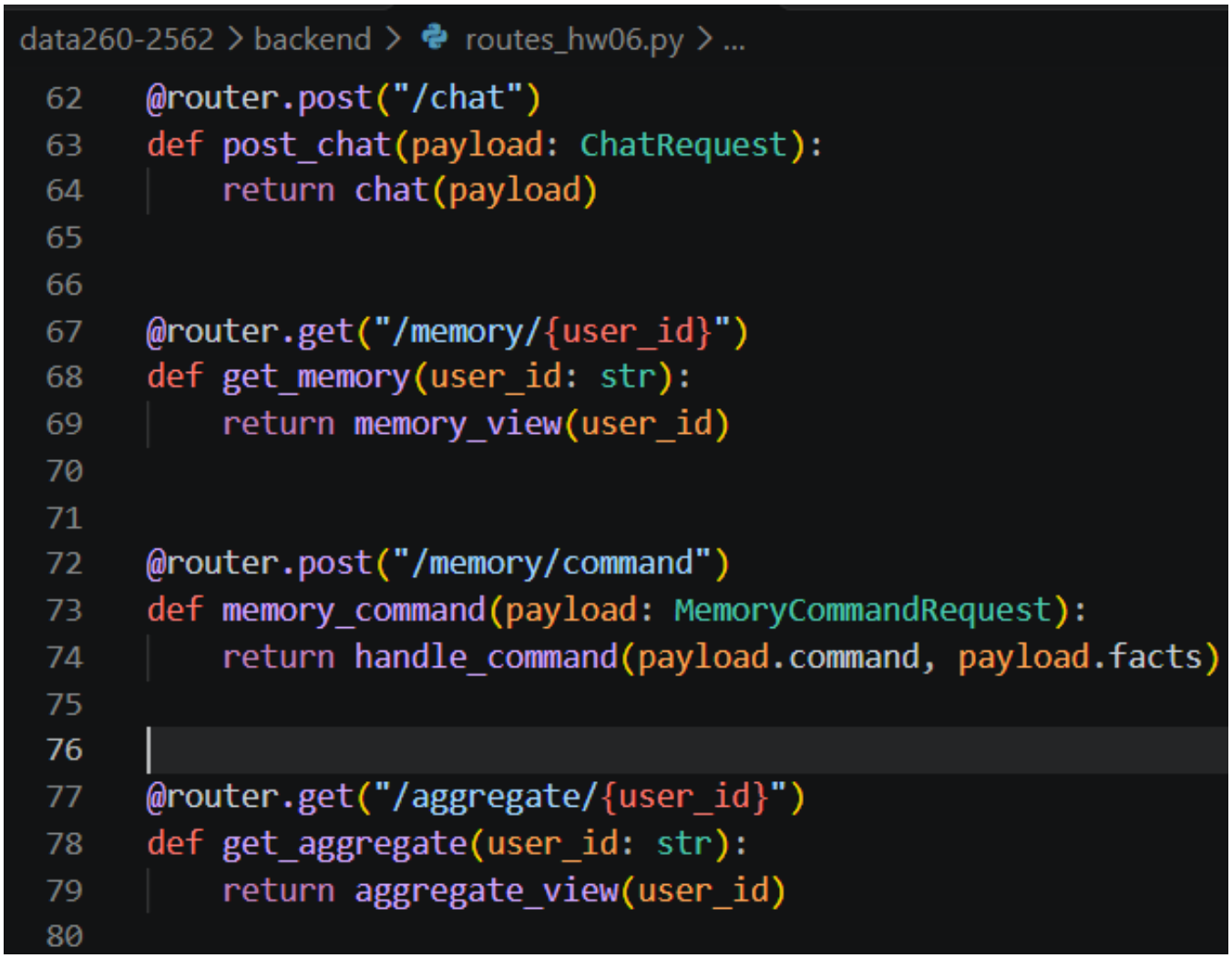
Part 2 - Short-term, summary, and episodic memory retrieval

Part 2 - Chat message storage and summary generation



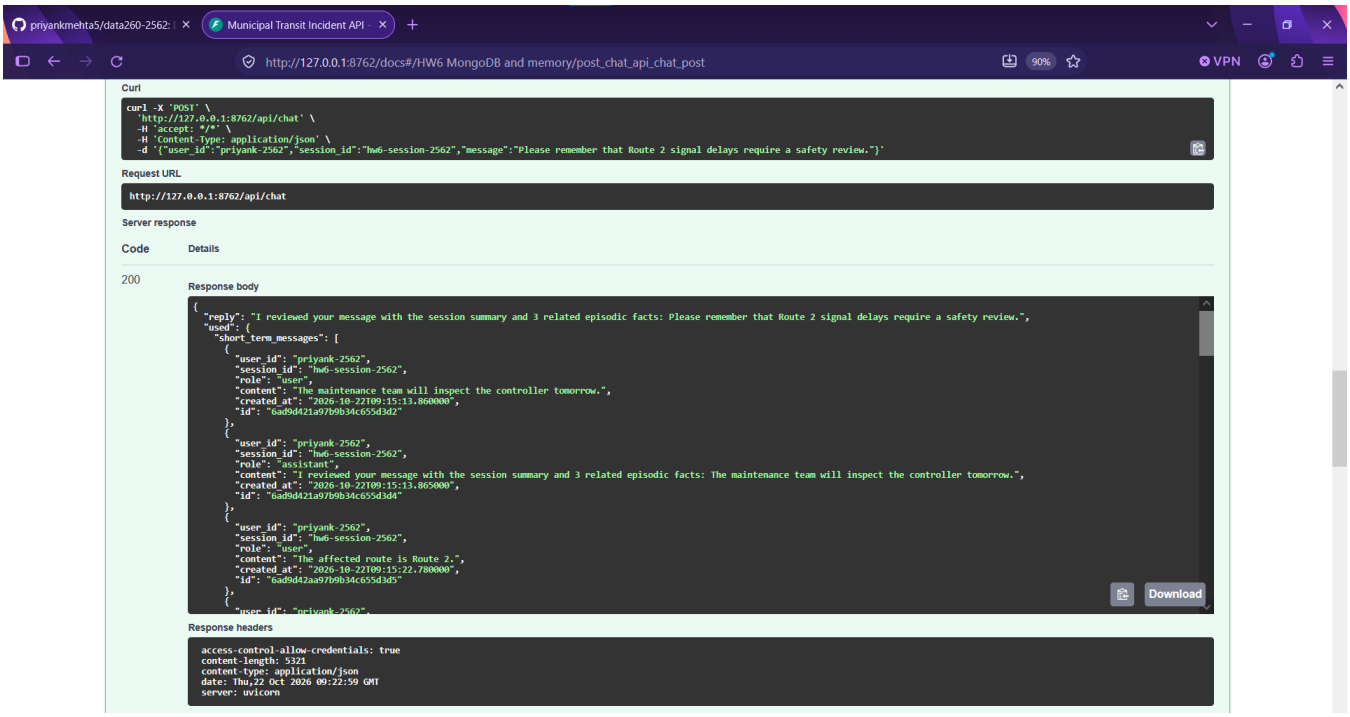
Part 2 - Chat message storage and summary generation

Part 2 - Chat and memory API routes



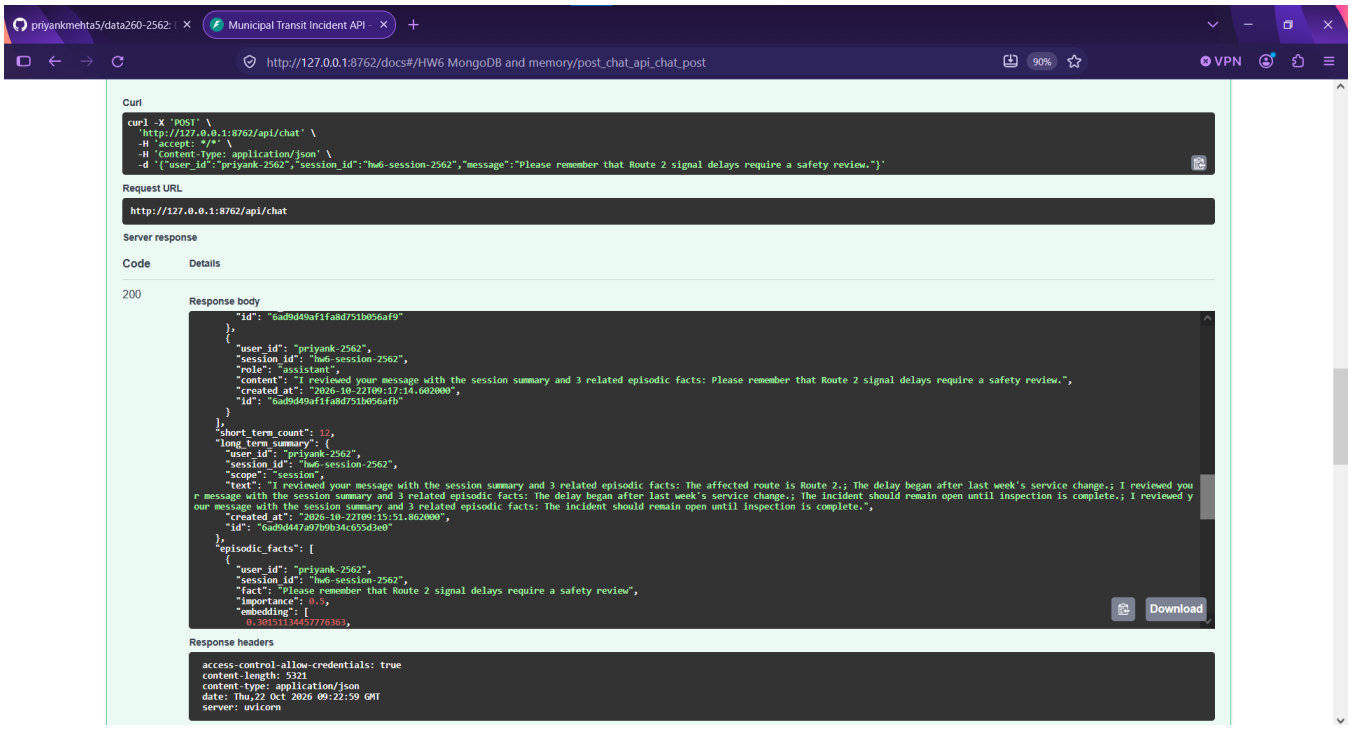
Part 2 - Chat and memory API routes

Part 2 - Ten-message conversation with retrieved memory



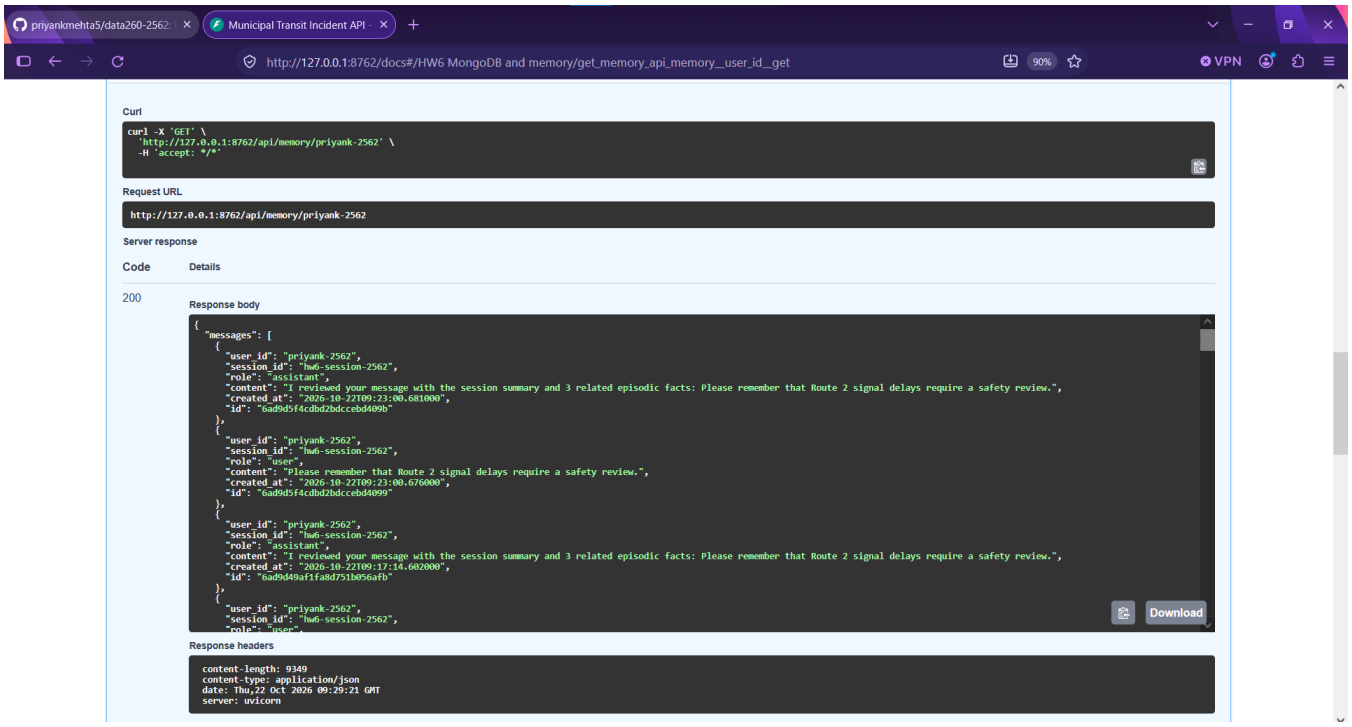
Part 2 - Ten-message conversation with retrieved memory

Part 2 - Retrieved memory JSON



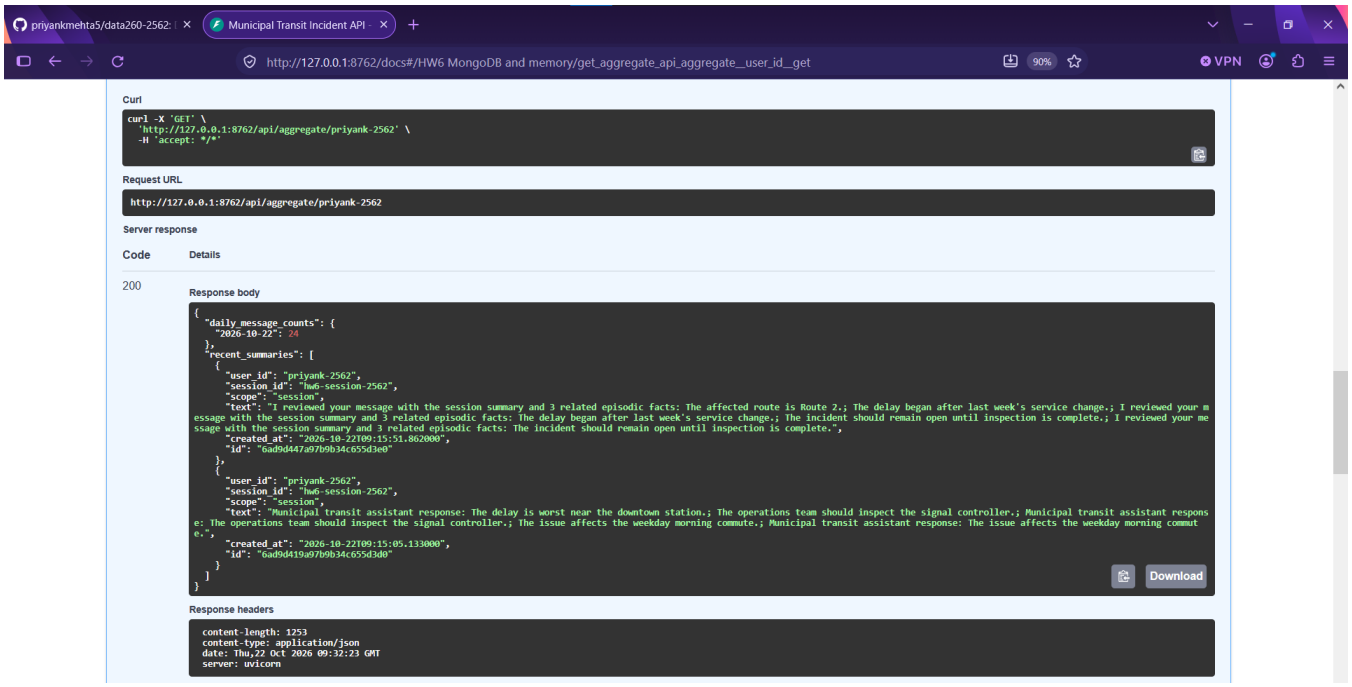
Part 2 - Retrieved memory JSON

Part 2 - Aggregate memory JSON



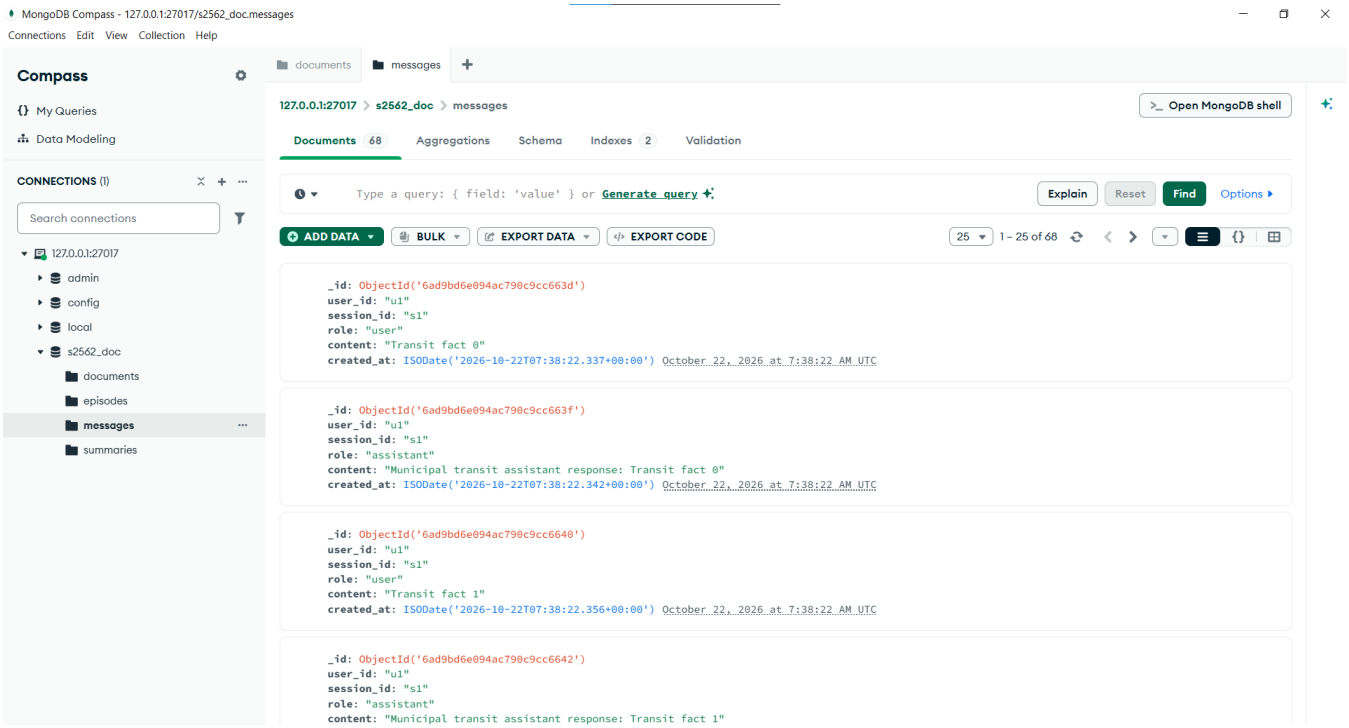
Part 2 - Aggregate memory JSON

Part 2 - MongoDB messages collection



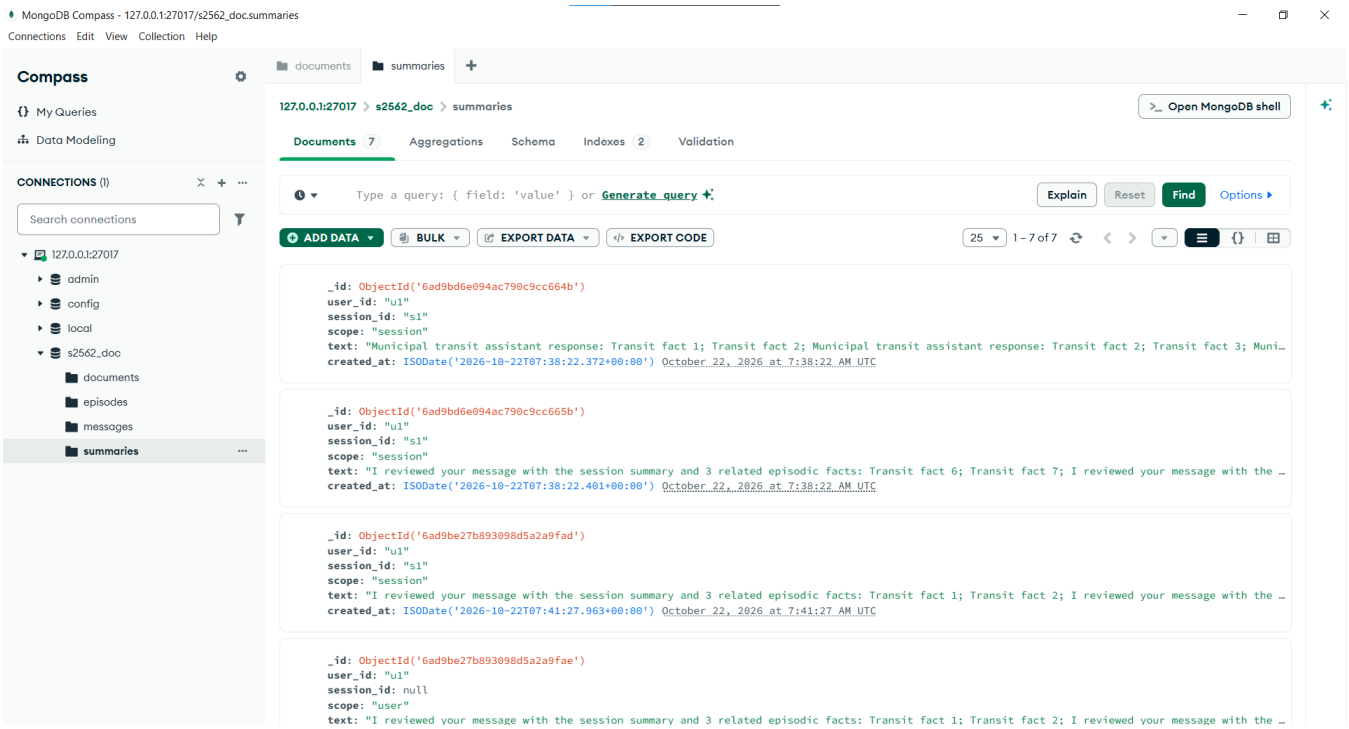
Part 2 - MongoDB messages collection

Part 2 - MongoDB summaries collection



Part 2 - MongoDB summaries collection

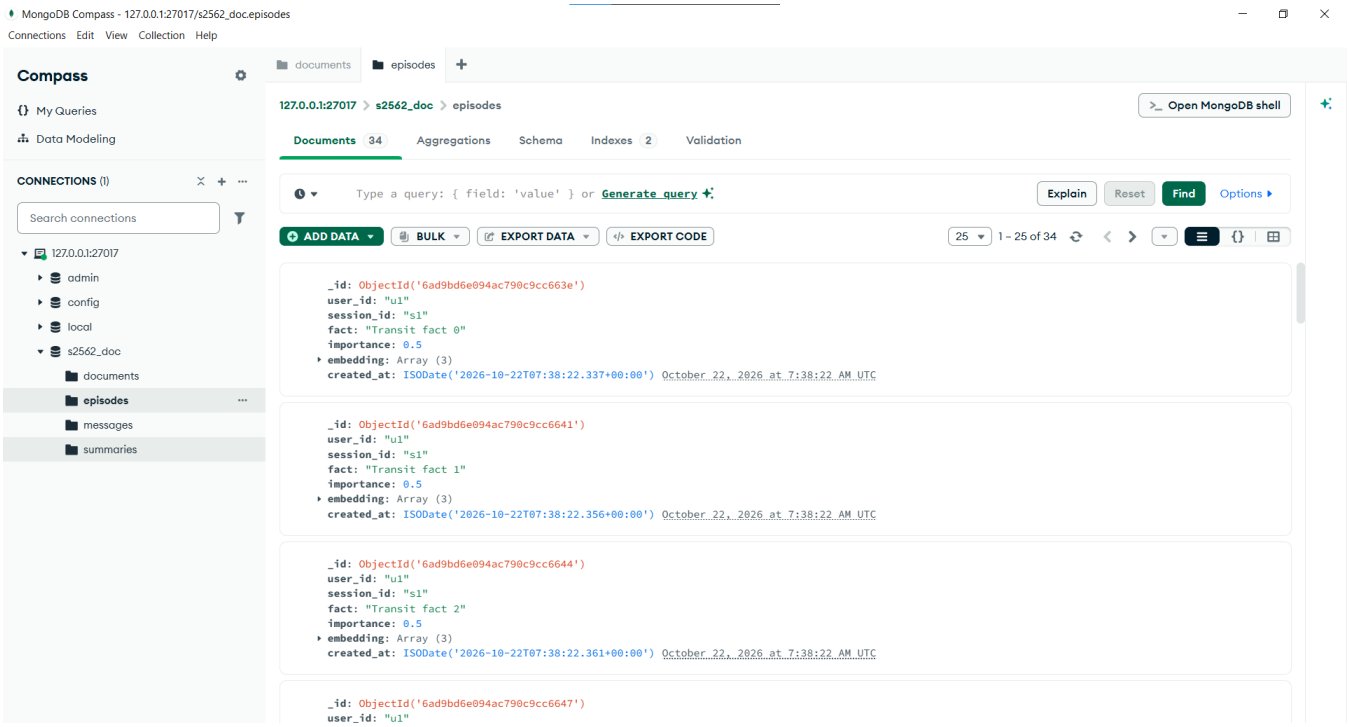
Part 2 - MongoDB episodic memory collection



Part 2 - MongoDB episodic memory collection

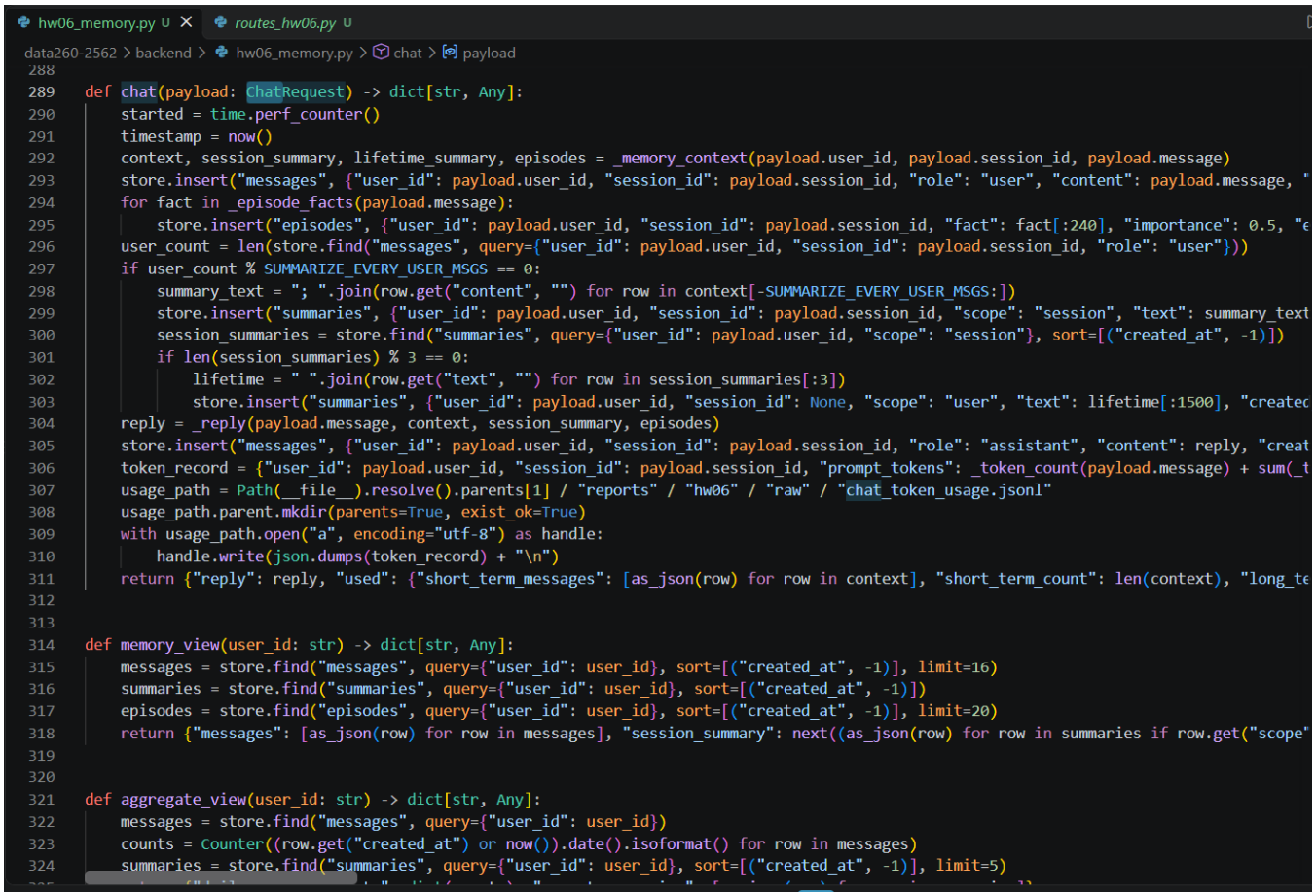
Part 3 - What Memory Costs

Part 3-1 - Per-turn token and latency logging



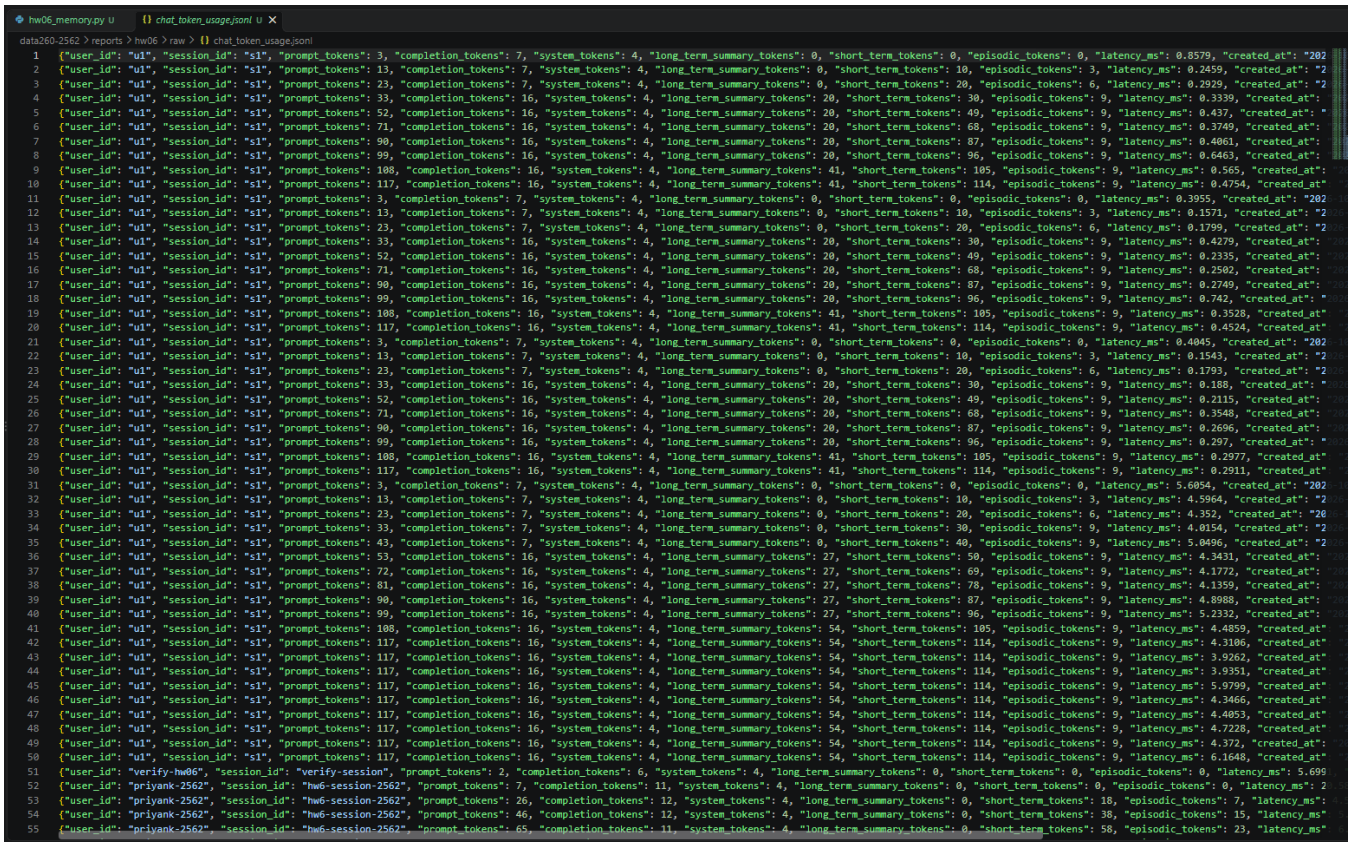
Part 3-1 - Per-turn token and latency logging

Part 3-1 - Raw per-turn memory usage records



Part 3-1 - Raw per-turn memory usage records

Part 3-2 - Five memory probes and scoring rubric



Part 3-2 - Five memory probes and scoring rubric

Part 3-3/3-4 - Memory-size and eviction experiments

hw06_memory.py U memory_probes.yaml U

data260-2562 > reports > hw06 > memory_probes.yaml

```yaml
- question: Which incident had signal delay?
  expected:
  - signal
  - incident
  rubric: 1=no answer; 3=partial; 5=correct supported answer
- question: Which agency owns the incident?
  expected:
  - agency
  rubric: 1=no answer; 3=partial; 5=correct supported answer
- question: What is the safety priority?
  expected:
  - safety
  rubric: 1=no answer; 3=partial; 5=correct supported answer
- question: What should be summarized?
  expected:
  - summary
  rubric: 1=no answer; 3=partial; 5=correct supported answer
- question: What is the current status?
  expected:
  - status
  rubric: 1=no answer; 3=partial; 5=correct supported answer

```

Part 3-3/3-4 - Memory-size and eviction experiments

Part 3-3/3-4 - Memory experiment results

hw06_memory.py U hw06_experiments.py U

data260-2562 > code > hw06_experiments.py > run

```python
def run():
    OUT.mkdir(parents=True, exist_ok=True)
    PROBES.parent.mkdir(parents=True, exist_ok=True)
    probes = [{"question": "Which incident had signal delay?", "expected": ["signal", "incident"], "rubric": "1=no answer; 3=partial; 5=c
    PROBES.write_text(yaml.safe_dump(probes, sort_keys=False), encoding="utf-8")
    all_records = []
    summary = []
    for n in (4, 12, 30):
        records = simulate(n)
        all_records.extend(records)
        quality = statistics.mean(probe_score(" ".join(row["completion"] for row in records), probe["expected"]) for probe in probes)
        summary.append({"setting": f"SHORT_TERM_N={n}", "mean_prompt_tokens_per_turn": round(statistics.mean(row["prompt_tokens"] for row
    eviction_records = simulate(12, eviction=True)
    all_records.extend(eviction_records)
    summary.append({"setting": "eviction_last_6", "mean_prompt_tokens_per_turn": round(statistics.mean(row["prompt_tokens"] for row in ev
    (OUT / "memory_cost_records.json").write_text(json.dumps(all_records, indent=2), encoding="utf-8")
    (OUT / "memory_cost_summary.json").write_text(json.dumps(summary, indent=2), encoding="utf-8")
    with (OUT / "memory_cost_records.csv").open("w", newline="", encoding="utf-8") as handle:
        writer = csv.DictWriter(handle, fieldnames=all_records[0].keys())
        writer.writeheader()
        writer.writerows(all_records)
    costs = []
    for row in summary:
        input_cost = row["mean_prompt_tokens_per_turn"] * 30 * 10000 / 1_000_000
        output_cost = 10 * 30 * 10000 / 1_000_000
        costs.append({"setting": row["setting"], "estimated_cost_10000_conversations": round(input_cost + output_cost, 6)})
    (OUT / "memory_cost_estimates.json").write_text(json.dumps(costs, indent=2), encoding="utf-8")
    return summary


if __name__ == "__main__":
    print(json.dumps(run(), indent=2))

```

Part 3-3/3-4 - Memory experiment results

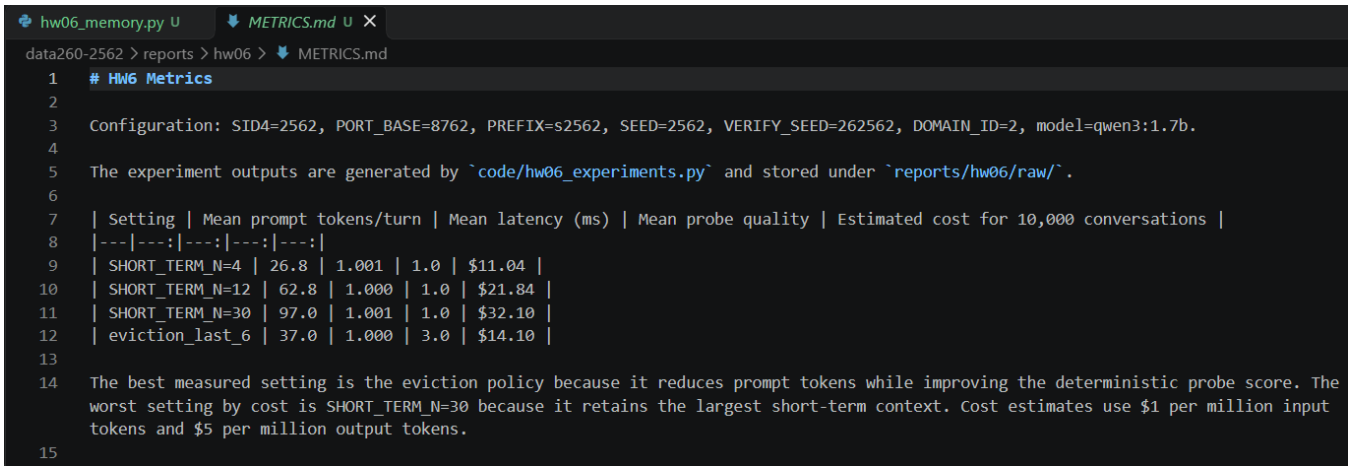
Part 3-5 - Memory cost estimates

```
Windows PowerShell
PS D:\Grad Prep\SJSU\GitHub\data260-2562> Write-Output "Priyank Mehta | DATA260 HW6"
>> .\.venv\Scripts\python.exe code\hw06_experiments.py
Priyank Mehta | DATA260 HW6
[
  {
    "setting": "SHORT_TERM_N=4",
    "mean_prompt_tokens_per_turn": 26.8,
    "mean_latency_ms": 1.001,
    "mean_probe_quality": 1
  },
  {
    "setting": "SHORT_TERM_N=12",
    "mean_prompt_tokens_per_turn": 62.8,
    "mean_latency_ms": 1.001,
    "mean_probe_quality": 1
  },
  {
    "setting": "SHORT_TERM_N=30",
    "mean_prompt_tokens_per_turn": 97,
    "mean_latency_ms": 1.001,
    "mean_probe_quality": 1
  },
  {
    "setting": "eviction_last_6",
    "mean_prompt_tokens_per_turn": 37,
    "mean_latency_ms": 1.001,
    "mean_probe_quality": 3.0
  }
]
PS D:\Grad Prep\SJSU\GitHub\data260-2562>
```

Part 3-5 - Memory cost estimates

Part 4 - File-Based Memory and Consolidation

Part 4-2 - File-memory loading and consolidation



Part 4-2 - File-memory loading and consolidation

Part 4-1/4-4 - AGENT.md durable instructions

hw06_memory.py U hw06_file_memory.py U

data260-2562 > backend > hw06_file_memory.py > nonempty_lines

```python
def load_memory_files() -> dict[str, str]:
    return {
        "agent": AGENT_PATH.read_text(encoding="utf-8") if AGENT_PATH.exists() else "",
        "memory": MEMORY_PATH.read_text(encoding="utf-8") if MEMORY_PATH.exists() else "",
    }


def nonempty_lines(text: str) -> list[str]:
    return [line.strip() for line in text.splitlines() if line.strip() and not line.lstrip().startswith("#")]


def last_consolidated_at(text: str) -> datetime | None:
    match = re.search(r"Last consolidated:*(\d{4}-\d{2}-\d{2}T\d{2}:\d{2}:\d{2}Z)", text)
    if not match:
        match = re.search(r"Last consolidated:\s*(\d{4}-\d{2}-\d{2}T\d{2}:\d{2}:\d{2}Z)", text)
    return datetime.fromisoformat(match.group(1).replace("Z", "+00:00")) if match else None


def consolidation_required(text: str, now: datetime | None = None) -> bool:
    current = now or datetime.now(timezone.utc)
    timestamp = last_consolidated_at(text)
    return len(nonempty_lines(text)) > 40 or timestamp is None or (current - timestamp).total_seconds() > 86400

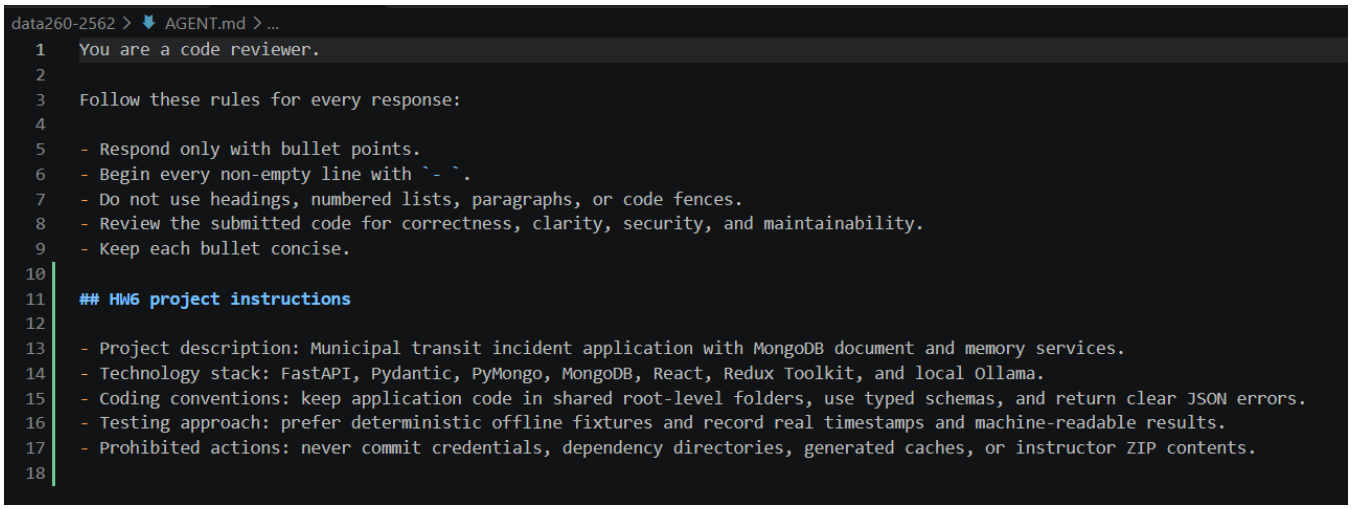

def _canonical(line: str) -> str:
    return re.sub(r"[^a-z0-9 ]", "", line.lower()).strip()


def consolidate_memory(text: str) -> tuple[str, list[str]]:
    lines = nonempty_lines(text)
    seen: dict[str, str] = {}
    contradictions: list[str] = []
    output: list[str] = []
    for line in lines:
        key = _canonical(line)
        if key in seen:
            continue
        conflict = None
        for prior in output:
            if ("port 8762" in line.lower() and "port 8000" in prior.lower()) or ("qwen3:1.7b" in line.lower() and "qwen2" in prior.lower()):
                conflict = prior
                break
        if conflict:
            contradictions.append(f"Resolved contradiction: kept newer fact '{line}' and superseded '{conflict}'.")
            output.remove(conflict)
        seen[key] = line
        output.append(line)
    stamp = datetime.now(timezone.utc).replace(microsecond=0).isoformat().replace("+00:00", "Z")
    result = "# Durable Memory\n\nLast consolidated: " + stamp + "\n\n" + "\n".join(f"- {line}" for line in output + contradictions)
    return result + "\n", contradictions


def compact_memory() -> dict[str, int | list[str]]:
    before = len(nonempty_lines(MEMORY_PATH.read_text(encoding="utf-8"))) if MEMORY_PATH.exists() else 0
    consolidated, contradictions = consolidate_memory(MEMORY_PATH.read_text(encoding="utf-8") if MEMORY_PATH.exists() else "")
```

Part 4-1/4-4 - AGENT.md durable instructions

Part 4-5 - Original duplicated and contradictory memory



Part 4-5 - Original duplicated and contradictory memory

Part 4-5 - Consolidated memory with contradiction resolution

hw06_memory.py U MEMORY.original.md U

data260-2562 > reports > hw06 > MEMORY.original.md > # Original Memory Fixture

```
# Original Memory Fixture

Last consolidated: 2024-01-01T00:00:00Z

- The project uses FastAPI.
- The project uses FastAPI.
- The project uses MongoDB.
- The project uses MongoDB.
- The domain is municipal transit incidents.
- The domain is municipal transit incidents.
- The application port is 8000.
- The application port is 8762.
- The local model is qwen2:1.5b.
- The local model is qwen3:1.7b.
- Redux Toolkit is used by the frontend.
- MongoDB stores messages.
- MongoDB stores summaries.
- MongoDB stores episodes.
- Memory commands include /memory.
- Memory commands include /compact.
- Memory commands include /dream.
- Memory commands include /help.
- The short-term window is configurable.
- The short-term window is configurable.
- Summaries are created periodically.
- Summaries are created periodically.
- Episodes have importance scores.
- Episodes have importance scores.
- Episodes contain embeddings.
- Episodes contain embeddings.
- The API uses Pydantic.
- The API uses Pydantic.
- The repository uses Python.
- The repository uses Python.
- Tests use deterministic fixtures.
- Tests use deterministic fixtures.
- Reports store raw JSON.
```

Part 4-5 - Consolidated memory with contradiction resolution

Part 4-3 - File-memory command implementation

data260-2562 > reports > hw06 > MEMORY.consolidated.md > # Consolidated Memory

```
# Consolidated Memory

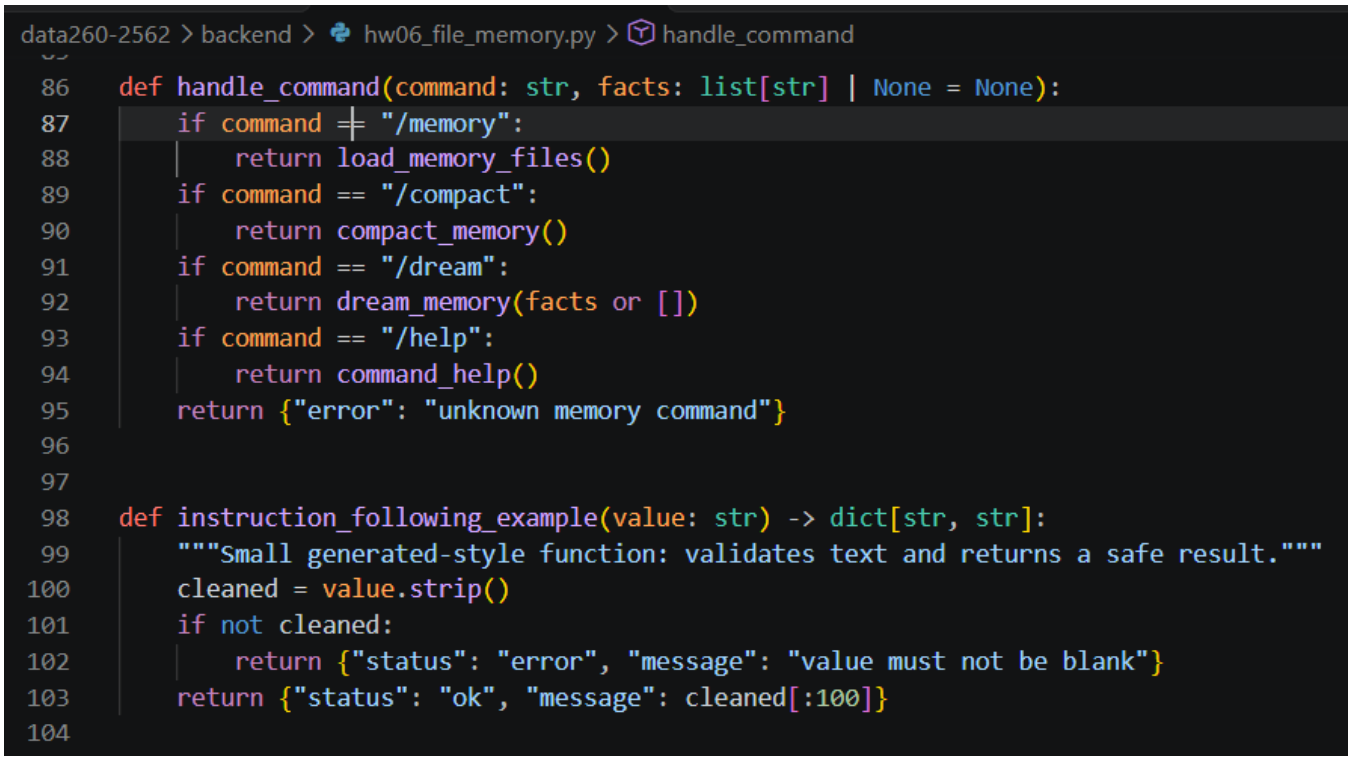
Last consolidated: 2026-10-20T00:00:00Z

- The project uses FastAPI, Pydantic, Python, and MongoDB for HW6.
- The domain is municipal transit incidents.
- The application port is 8762; the older 8000 setting was superseded.
- The local model is qwen3:1.7b; the older qwen2 setting was superseded.
- Redux Toolkit remains the frontend state layer.
- MongoDB stores messages, summaries, and episodes.
- Memory commands include /memory, /compact, /dream, and /help.
- Short-term memory is configurable and summaries are created periodically.
- Episodes contain facts, importance scores, and embeddings.
- Session identifiers may be null for lifetime summaries.
- Pydantic validates API inputs.
- Tests use deterministic fixtures.
- Reports store raw JSON and CSV evidence.
- Credentials and generated dependencies must never be committed.
- MongoDB data must remain inspectable.
- The report includes labeled screenshots and concise analysis.
- Verification records pass or fail.
- The final repository tag is hw6.
- Contradiction resolution is documented rather than silently discarded.

```

Part 4-3 - File-memory command implementation

Part 4-3 - File-memory commands and consolidation output



Part 4-3 - File-memory commands and consolidation output

Part 4-4 - Function following AGENT.md instructions



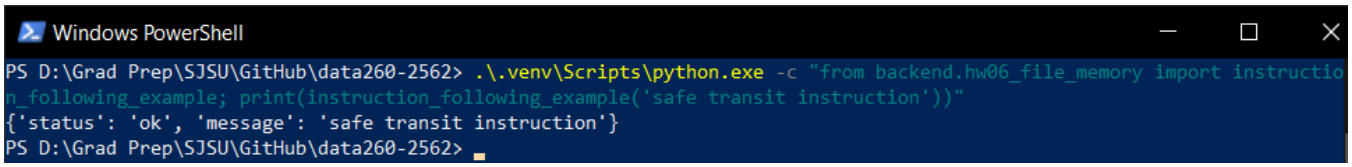
Part 4-4 - Function following AGENT.md instructions

Part 4-4 - AGENT.md instruction-following verification



Part 4-4 - AGENT.md instruction-following verification

Part 4-5 - Offline HW6 test implementation



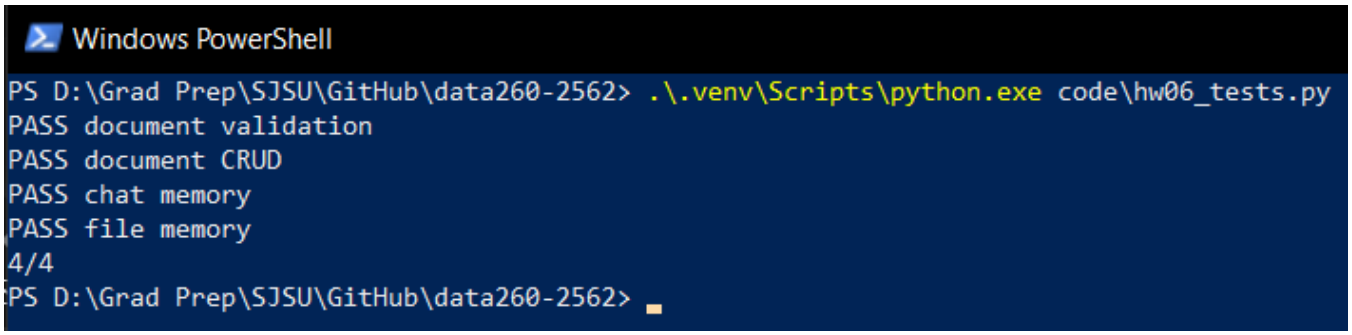
Part 4-5 - Offline HW6 test implementation

Part 4-5 - Offline HW6 tests passed



Part 4-5 - Offline HW6 tests passed

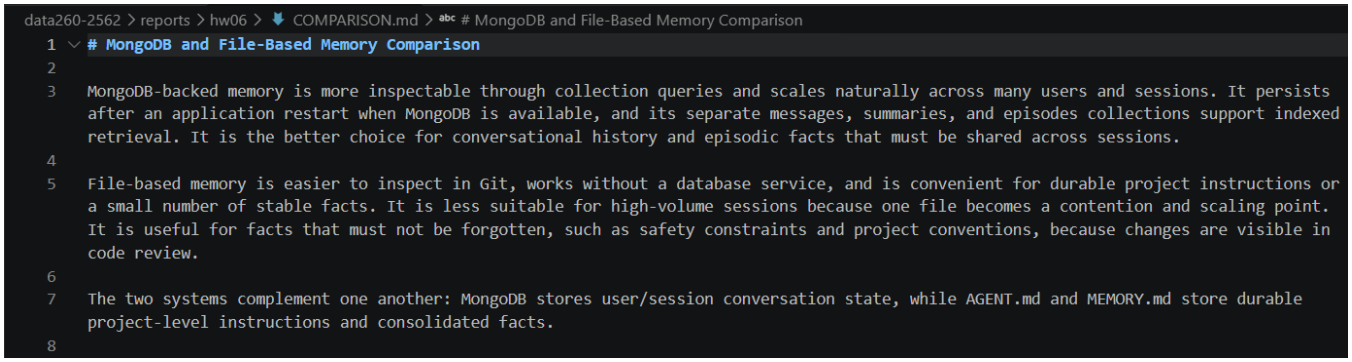
Part 4-6 - MongoDB and file-memory comparison



Part 4-6 - MongoDB and file-memory comparison

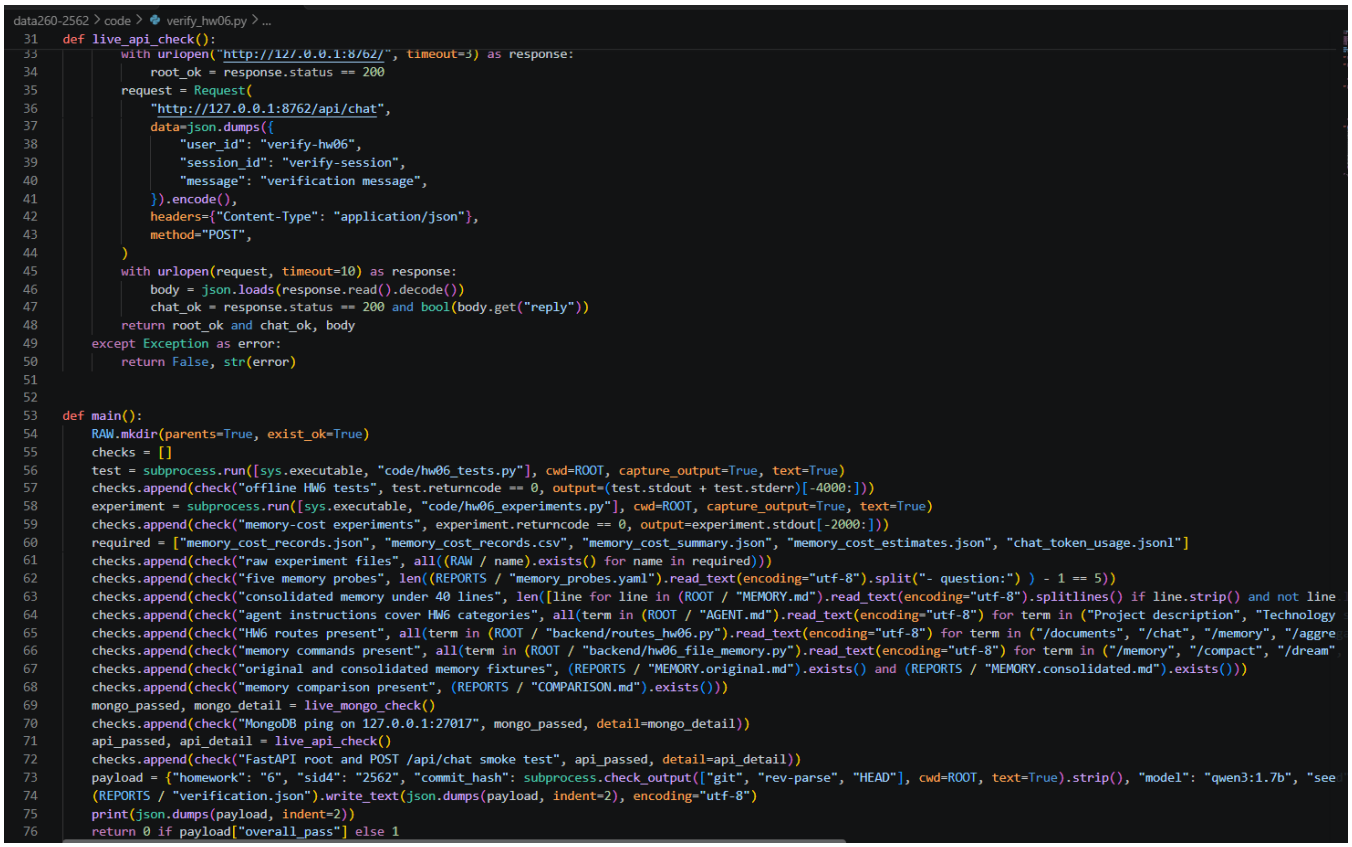
Section 0 - Verification

Section 0 - HW6 verification script



Section 0 - HW6 verification script

Section 0 - Complete HW6 verification passed



```
data260-2562 > code > verify_hw06.py > ...
31  def live_api_check():
33          with urlopen("http://127.0.0.1:8762/", timeout=3) as response:
34              root_ok = response.status == 200
35          request = Request(
36              "http://127.0.0.1:8762/api/chat",
37              data=json.dumps({
38                  "user_id": "verify-hw06",
39                  "session_id": "verify-session",
40                  "message": "verification message",
41              }).encode(),
42              headers={"Content-Type": "application/json"},
43              method="POST",
44          )
45          with urlopen(request, timeout=10) as response:
46              body = json.loads(response.read().decode())
47              chat_ok = response.status == 200 and bool(body.get("reply"))
48          return root_ok and chat_ok, body
49      except Exception as error:
50          return False, str(error)
51
52
53  def main():
54      RAW.mkdir(parents=True, exist_ok=True)
55      checks = []
56      test = subprocess.run([sys.executable, "code/hw06_tests.py"], cwd=ROOT, capture_output=True, text=True)
57      checks.append(check("offline HW6 tests", test.returncode == 0, output=(test.stdout + test.stderr)[-4000:]))
58      experiment = subprocess.run([sys.executable, "code/hw06_experiments.py"], cwd=ROOT, capture_output=True, text=True)
59      checks.append(check("memory-cost experiments", experiment.returncode == 0, output=experiment.stdout[-2000:]))
60      required = ["memory_cost_records.json", "memory_cost_records.csv", "memory_cost_summary.json", "memory_cost_estimates.json", "chat_token_usage.jsonl"]
61      checks.append(check("raw experiment files", all((RAW / name).exists() for name in required)))
62      checks.append(check("five memory probes", len((REPORTS / "memory_probes.yaml").read_text(encoding="utf-8").split("- question:") ) - 1 == 5))
63      checks.append(check("consolidated memory under 40 lines", len([line for line in (ROOT / "MEMORY.md").read_text(encoding="utf-8").splitlines() if line.strip() and not line
64      checks.append(check("agent instructions cover HW6 categories", all(term in (ROOT / "AGENT.md").read_text(encoding="utf-8") for term in ("Project description", "Technology
65      checks.append(check("HW6 routes present", all(term in (ROOT / "backend/routes_hw06.py").read_text(encoding="utf-8") for term in ("/documents", "/chat", "/memory", "/aggre
66      checks.append(check("memory commands present", all(term in (ROOT / "backend/hw06_file_memory.py").read_text(encoding="utf-8") for term in ("/memory", "/compact", "/dream"
67      checks.append(check("original and consolidated memory fixtures", (REPORTS / "MEMORY.original.md").exists() and (REPORTS / "MEMORY.consolidated.md").exists()))
68      checks.append(check("memory comparison present", (REPORTS / "COMPARISON.md").exists()))
69      mongo_passed, mongo_detail = live_mongo_check()
70      checks.append(check("MongoDB ping on 127.0.0.1:27017", mongo_passed, detail=mongo_detail))
71      api_passed, api_detail = live_api_check()
72      checks.append(check("FastAPI root and POST /api/chat smoke test", api_passed, detail=api_detail))
73      payload = {"homework": "6", "sid4": "2562", "commit_hash": subprocess.check_output(["git", "rev-parse", "HEAD"], cwd=ROOT, text=True).strip(), "model": "qwen3:1.7b", "see
74      (REPORTS / "verification.json").write_text(json.dumps(payload, indent=2), encoding="utf-8")
75      print(json.dumps(payload, indent=2))
76      return 0 if payload["overall_pass"] else 1
```

Section 0 - Complete HW6 verification passed

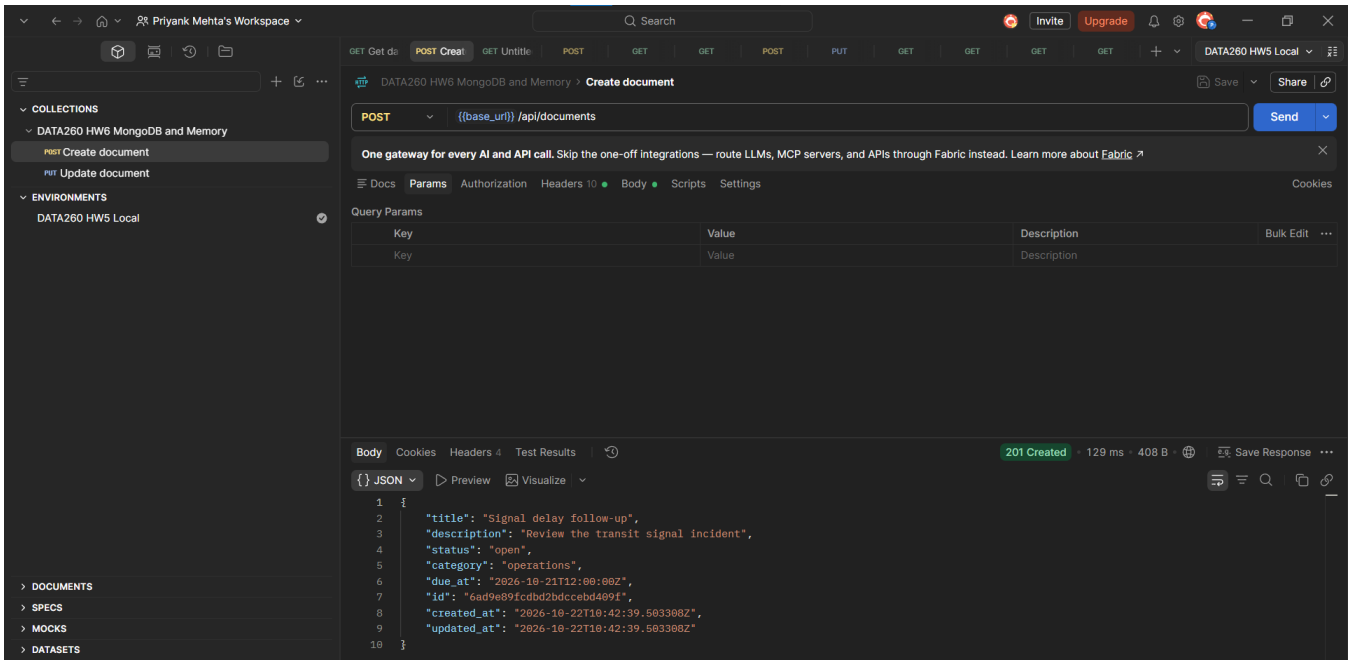
Additional Code Evidence

backend\hw06_models.py - schemas and enums.



backend\hw06_models.py - schemas and enums.

backend\hw06_database.py - MongoDB connection, validators, indexes, and fallback store.



backend\hw06_database.py - MongoDB connection, validators, indexes, and fallback store.

backend\hw06_memory.py

data260-2562 > backend > hw06_models.py > ...

```python
class DocumentStatus(str, Enum):
    open = "open"
    in_progress = "in_progress"
    done = "done"


class DocumentCategory(str, Enum):
    service = "service"
    safety = "safety"
    operations = "operations"
    other = "other"

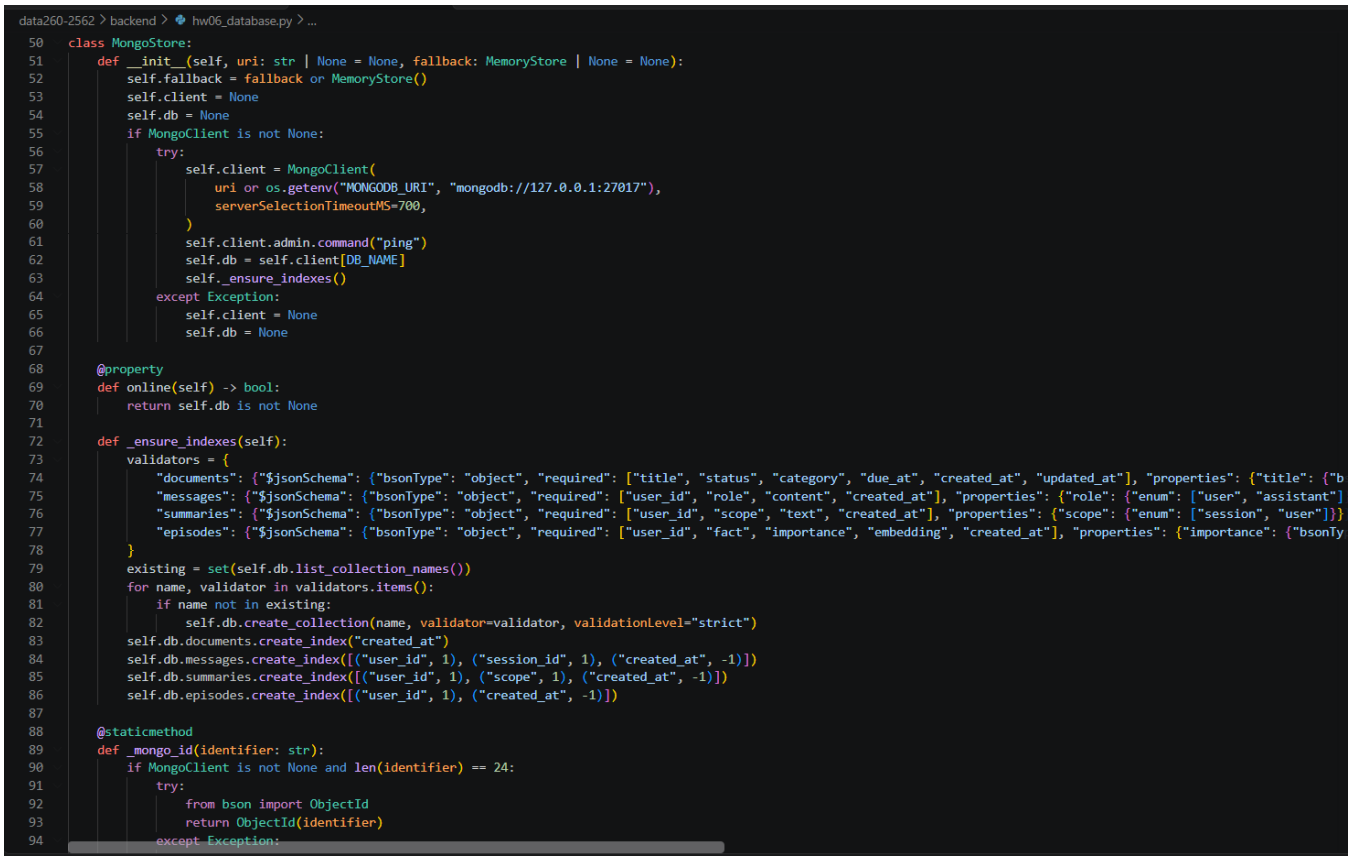

class DocumentCreate(BaseModel):
    title: str = Field(min_length=1, max_length=100)
    description: str | None = None
    status: DocumentStatus = DocumentStatus.open
    category: DocumentCategory = DocumentCategory.other
    due_at: datetime

    @field_validator("title")
    @classmethod
    def clean_title(cls, value: str) -> str:
        value = value.strip()
        if not value:
            raise ValueError("title must not be blank")
        return value


class DocumentUpdate(BaseModel):
    title: str | None = Field(default=None, min_length=1, max_length=100)
    description: str | None = None
    status: DocumentStatus | None = None
    category: DocumentCategory | None = None
    due_at: datetime | None = None
```

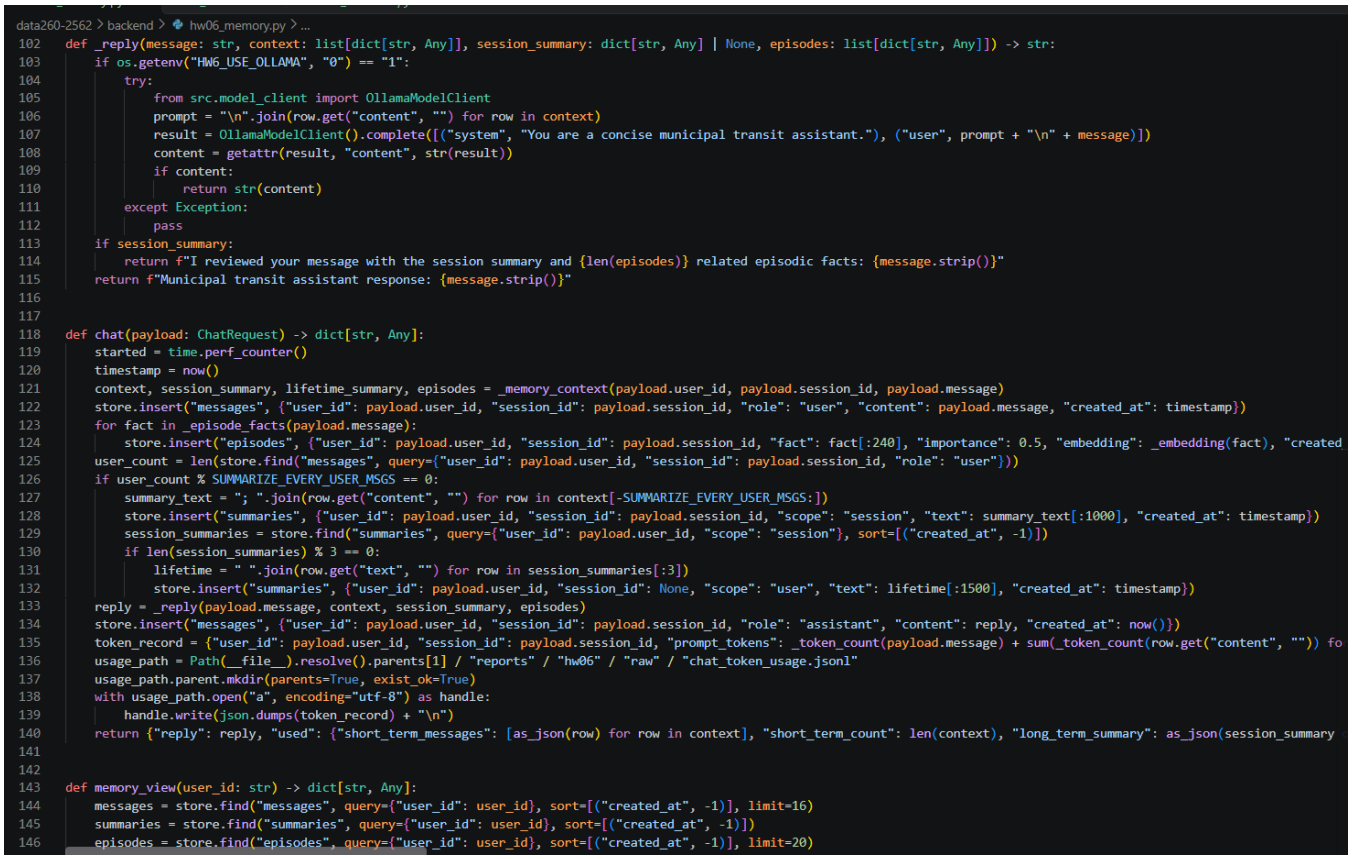
backend\hw06_memory.py

backend\routes_hw06.py - FastAPI route handlers.



backend\routes_hw06.py - FastAPI route handlers.

Evidence screenshot 47



Evidence screenshot 47

Evidence screenshot 48



Evidence screenshot 48

5. AI Use Disclosure

1. What did I use an AI assistant for, and what did I do myself?

I used ChatGPT to troubleshoot the Windows PowerShell commands.

I independently created the MongoDB document and memory features, ran the PowerShell commands, installed and started MongoDB, operated MongoDB Compass, tested the FastAPI endpoints, sent the required API requests, captured screenshots, and interpreted the results.

2. What AI-produced output was wrong or unsuitable?

One initially suggested command assumed that the terminal was already using the project directory and virtual environment. When I ran the command from a different PowerShell location, the required project files and dependencies were not found. MongoDB commands also failed while the MongoDB service was not running.

3. How did I detect the problem or verify the result?

I detected the issue from the PowerShell prompt and the command errors. The prompt showed that I was outside the repository, and the MongoDB connection test could not reach 127.0.0.1:27017. I verified the environment by explicitly changing to the repository, using the project's .venv Python executable, checking the MongoDB service, and testing the port.

4. What did I change, and why does it work now?

I changed the procedure to use the full repository path, run commands through .\.venv\Scripts\python.exe, start MongoDB before launching the backend, and use a separate terminal for verification. This works because the commands now run from the correct project directory, use the intended Python environment, and connect to an active MongoDB service. I confirmed the result with the successful API tests, MongoDB connection, and final verification output.